

**Федеральное государственное автономное образовательное учреждение высшего образования  
«Московский государственный технический университет имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)**

КАФЕДРА Программное обеспечение ЭВМ, информационные технологии, ИУК4

Разработка программного обеспечения для обработки и визуализации  
телеметрии летательных аппаратов

Е.В. Красавин  
(инициалы, фамилия)

2026 год

УТВЕРЖДАЮ

Заведующий кафедрой

ИУК4

(индекс)

Ю.Е. Гагарин

(подпись)

30.11.2025

(дата)

**ЗАДАНИЕ**  
**на выполнение выпускной квалификационной работы**

Студент группы: Боков Артём Алексеевич

(фамилия, имя, отчество)

Тема выпускной квалификационной работы Разработка программного обеспечения для  
обработки и визуализации телеметрии летательных аппаратов

При выполнении ВКР:

|    | Используются / Не используются                                                                                                     | Да/Нет |
|----|------------------------------------------------------------------------------------------------------------------------------------|--------|
| 1) | Литературные источники и документы, имеющие гриф секретности                                                                       | Нет    |
| 2) | Литературные источники и документы, имеющие пометку «Для служебного пользования», иных пометок, запрещающих открытое опубликование | Нет    |
| 3) | Служебные материалы других организаций                                                                                             | Нет    |
| 4) | Результаты НИР (ОКР), выполняемой в МГТУ им. Н.Э. Баумана                                                                          | Нет    |
| 5) | Материалы по незавершенным исследованиям или материалы по завершенным исследованиям, но ещё не опубликованные в открытой печати    | Нет    |

|                                                                      |                               |
|----------------------------------------------------------------------|-------------------------------|
| Тема квалификационной работы утверждена распоряжением по факультету: |                               |
| Название факультета:                                                 | Информатика и управление, ИУК |
| Дата и рег. номер распоряжения:                                      | 30.11.2025 № 07.03.01-03/148  |

**Часть 1. Анализ требований и технологий разработки программного обеспечения**

Разработать техническое задание, провести анализ существующих технологий для разработки, обосновать выбор используемых инструментов, платформы для разработки, разработки и библиотек, определиться с необходимым функционалом приложения

## **Часть 2. Проектирование программного обеспечения**

Спроектировать и разработать архитектуру приложения, оптимальный способ хранения данных, информационную модель системы и протокол передачи данных для взаимодействия между компонентами системы

---

---

---

## **Часть 3. Контроль качества и интеграция компонентов программного обеспечения**

Выполнить тестирование работоспособности системы на всех браузерах, проверить корректность работы функционала приложения, осуществить проверку всех вероятных сценариев использования, произвести нагрузочное тестирование приложения. Разработать руководство для пользователя с описанием интерфейса программного продукта

---

---

---

### ***Оформление квалификационной работы:***

Расчетно-пояснительная записка на 61 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.):

- 
1. Актуальность. 2. Цель и задачи. 3. Функциональные требования к системе. 4. Описание приложения.
- 
5. Анализ существующих решений. 6. Инструменты разработки приложения. 7. Архитектура приложения.
- 
8. Архитектура клиентской части. 9. Архитектура серверной части.. 10. Диаграмма действий пользователя. 11. Диаграмма классов. 12. Контекстная диаграмма IDEF0.
- 
13. Диаграмма декомпозиции IDEF0. 14. Учебный режим веб-приложения. 15. Инженерный режим веб-приложения. 16. Режим воспроизведения сеанса. 17. Тёмный режим оформления.
- 

Дата выдачи задания 30.11.2025.

В соответствии с учебным планом выпускную квалификационную работу выполнить в полном объеме в срок до 30.05.2026г.

|                                      |           |                     |            |
|--------------------------------------|-----------|---------------------|------------|
| Руководитель квалификационной работы | <hr/>     | Е.В. Широкова       | 30.11.2025 |
|                                      | (подпись) | (инициалы, фамилия) | (дата)     |
| Студент                              | <hr/>     | А.А. Боков          | 30.11.2025 |
|                                      | (подпись) | (инициалы, фамилия) | (дата)     |

Примечание: Задание оформляется в двух экземплярах: один выдается обучающемуся, второй хранится на кафедре.

**Министерство науки и высшего образования Российской Федерации**

**Федеральное государственное автономное образовательное учреждение высшего образования  
«Московский государственный технический университет имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)**

|           |          |                     |              |
|-----------|----------|---------------------|--------------|
| ФАКУЛЬТЕТ | ИУК      |                     | УТВЕРЖДАЮ    |
| КАФЕДРА   | ИУК4     |                     |              |
| ГРУППА    | ИУК4-82Б | Заведующий кафедрой | ИУК4         |
|           |          |                     | (индекс)     |
|           |          | _____               | Ю.Е. Гагарин |
|           |          | (подпись)           | 30.11.2025   |
|           |          |                     | (дата)       |

### КАЛЕНДАРНЫЙ ПЛАН выполнения выпускной квалификационной работы

Студента Боков Артём Алексеевич

(фамилия, имя, отчество)

Тема квалификационной работы Разработка программного обеспечения для  
обработки и визуализации телеметрии летательных аппаратов

| №<br>п/п | Наименование этапов<br>выпускной квалификационной<br>работы    | Сроки выполнения этапов |      | Отметка о выполнении |                    |
|----------|----------------------------------------------------------------|-------------------------|------|----------------------|--------------------|
|          |                                                                | План                    | Факт | Должность            | Ф.И.О.,<br>подпись |
| 1.       | Задание на выполнение работы.<br>Формулирование проблемы, цели | 30.11.2025              |      | Руководитель ВКР     | Широкова<br>Е.В.   |
| 2.       | 1 часть                                                        | 19.05.2026              |      | Руководитель ВКР     | Широкова<br>Е.В.   |
| 3.       | 2 часть                                                        | 23.05.2026              |      | Руководитель ВКР     | Широкова<br>Е.В.   |
| 4.       | 3 часть                                                        | 28.05.2026              |      | Руководитель ВКР     | Широкова<br>Е.В.   |
| 5.       | 1-я редакция работы                                            | 29-30.05.2026           |      | Руководитель ВКР     | Широкова<br>Е.В.   |
| 6.       | Подготовка доклада и презентации                               | 01-02.06.2026           |      | Руководитель ВКР     | Широкова<br>Е.В.   |
| 7.       | Нормоконтроль                                                  | 03-04.06.2026           |      | Нормоконтролер       | Красавин Е.В.      |
| 8.       | Отзыв руководителя                                             | 05-06.06.2026           |      | Руководитель ВКР     | Широкова<br>Е.В.   |
| 9.       | Защита работы на ГЭК                                           | 15-22.06.2026           |      | Секретарь ГЭК        | Никитенко<br>У.В.  |

|                                      |           |                     |            |
|--------------------------------------|-----------|---------------------|------------|
| Руководитель квалификационной работы |           | Е.В. Широкова       | 30.11.2025 |
|                                      | (подпись) | (инициалы, фамилия) | (дата)     |
| Студент                              |           | А.А. Боков          | 30.11.2025 |
|                                      | (подпись) | (инициалы, фамилия) | (дата)     |

## **РЕФЕРАТ**

Расчетно-пояснительная записка 61 с., 29 рис., 12 таблиц, 30 источников.

Объектом разработки является клиент-серверное приложение для приёма, обработки и визуализации телеметрических данных летательных аппаратов.

Целью работы является разработка клиент-серверного приложения для мониторинга и визуализации телеметрии летательных аппаратов с поддержкой записи сеансов связи и экспорта отчётов в форматах PDF, CSV и JSON.

Поставленные задачи решаются путём проектирования и разработки серверной части на языке Go и веб-клиента на базе HTML/CSS/JavaScript, поддерживающих приём кадров телеметрии по TCP, передачу клиенту по WebSocket и выгрузку отчётов.

## СОДЕРЖАНИЕ

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                                  | 7  |
| 1. АНАЛИЗ ТРЕБОВАНИЙ И ТЕХНОЛОГИЙ РАЗРАБОТКИ<br>ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ .....  | 9  |
| 1.1. Основные требования .....                                                  | 9  |
| 1.2. Анализ предметной области и существующих решений.....                      | 10 |
| 1.3. Выбор средств разработки и технологий для клиентской части .....           | 13 |
| 1.4. Выбор средств разработки и технологий для серверной части .....            | 19 |
| 2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ .....                                | 23 |
| 2.1. Проектирование функциональной модели.....                                  | 23 |
| 2.2. Принцип работы приложения.....                                             | 25 |
| 2.3. Структура данных .....                                                     | 29 |
| 2.4. Алгоритм обработки телеметрии .....                                        | 37 |
| 3. КОНТРОЛЬ КАЧЕСТВА И ИНТЕГРАЦИЯ КОМПОНЕНТОВ<br>ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ..... | 41 |
| 3.1. Разработка тестов и результаты тестирования .....                          | 41 |
| 3.2. Интерфейс веб-приложения.....                                              | 44 |
| 3.3. Руководство пользователя.....                                              | 48 |
| 3.4. Руководство администратора .....                                           | 54 |
| ЗАКЛЮЧЕНИЕ .....                                                                | 57 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                                          | 58 |
| ПРИЛОЖЕНИЕ А .....                                                              | 61 |

## ВВЕДЕНИЕ

Актуальность работы обусловлена тем, что в современном обществе всё больше внимания уделяется использованию цифровых технологий для мониторинга, анализа и повышения эффективности управления сложными системами. При эксплуатации летательных аппаратов важную роль играет своевременное получение, обработка и визуализация телеметрических данных, позволяющих оценивать состояние оборудования, корректировать режимы работы и повышать надёжность полётов. Однако существующие подходы к обработке и представлению телеметрии часто не обеспечивают достаточную наглядность, устойчивость обработки данных и удобство получения отчётов пользователем. [3].

Поэтому актуальным является создание клиент-серверного приложения для обработки телеметрии летательных аппаратов, состоящего из компонентов, которые в совокупности обеспечивают приём, разбор, хранение, визуализацию и формирование отчётов по полученным данным. Такой подход позволяет повысить качество обратной связи для пользователей и упростить контроль эксплуатационных параметров летательного аппарата. Кроме того, интеграция механизмов группировки данных по сеансам связи и поддержка экспорта в форматы PDF, CSV и JSON делают систему более практичной и пригодной для дальнейшего развития.

Объектом выпускной квалификационной работы является разработка клиент-серверной системы обработки телеметрических данных летательных аппаратов и их визуализации для пользователей.

Предметом исследования работы является отрасль телеметрии, дистанционного контроля и визуализации данных, применяемая при мониторинге летательных аппаратов.

Целью работы является разработка клиент-серверного приложения для мониторинга и визуализации телеметрии летательных аппаратов с поддержкой экспорта и импорта отчётов.

Для достижения поставленной цели решаются следующие задачи:

1. Провести сравнительный анализ существующих подходов и систем аналогичного назначения;
2. Определить оптимальную структуру приложения и протокол обмена данными;
3. Реализовать программные компоненты системы приёма, обработки, хранения и визуализации телеметрии;
4. Осуществить тестирование компонентов системы и проверку корректности обработки данных;
5. Разработать сопроводительную документацию, включая описание протокола, архитектуры и сценариев работы.



# 1. АНАЛИЗ ТРЕБОВАНИЙ И ТЕХНОЛОГИЙ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

## 1.1. Основные требования

Рассмотрим основные функциональные требования к разрабатываемому приложению.

*Приём потока телеметрии по TCP; одно TCP-подключение соответствует одному сеансу связи.*

- Разбор входящих бинарных данных в структуру кадра телеметрии и выделение параметров.
- Хранение принятых кадров и сеансов в оперативной памяти в пределах заданных ограничений.

*Доставка данных веб-клиенту:*

- Передача декодированных кадров подключенным клиентам по WebSocket в реальном времени.

*Учебный режим (упрощённый просмотр):*

- Отображение упрощённого набора параметров кадра в виде таблицы.
- Визуализация ориентации аппарата (крен, тангаж, рысканье).
- Отображение положения аппарата и пройденного трека на карте.
- Построение графика температур (средняя, освещённая, теневая стороны).

*Инженерный режим (полный кадр):*

- Отображение полного кадра телеметрии (~35 параметров) по группам.
- Графики температур и параметров электропитания (напряжение АКБ, заряд АКБ).
- Резервное получение последнего полного кадра через REST-эндпоинт.
- Карта и блок ориентации, аналогичные учебному режиму.

### *Воспроизведение записанного сеанса:*

- Загрузка JSON-файла сеанса (экспортированного сервером) с управлением воспроизведением: воспроизвести, стоп, перемотка по кадрам, выбор скорости 0,25–8×, переключение учебного и инженерного вида.

### *Экспорт данных и отчётность:*

- Выгрузка отчётов и архивов: PDF, CSV учебного и полного формата, JSON сеансов.

### *Пользовательский интерфейс и развёртывание:*

- Переключение языка интерфейса: русский / английский.
- Переключение светлой и тёмной темы оформления.
- Хранение телеметрии в оперативной памяти с ограничением числа сеансов и числа кадров в одном сеансе.
- HTTP-сервер предоставляет статические файлы веб-интерфейса и API на порту 8080.
- Поддерживается запуск серверной части на типовых ОС (Windows / Linux) при наличии среды Go.
- Доступ к пользовательскому интерфейсу осуществляется через современный веб-браузер.

## **1.2. Анализ предметной области и существующих решений**

Разрабатываемое приложение предназначено для приёма и визуализации телеметрии летательного аппарата в реальном времени, а также для последующего анализа записанных сеансов связи.

Ключевыми объектами предметной области являются: сеанс связи (одно ТСР-подключение), кадр телеметрии и набор параметров кадра.

- В системе предусмотрены два представления данных: учебное (упрощённый набор параметров) и инженерное (полный кадр с группировкой параметров).

- Для последующего анализа реализован режим воспроизведения сеанса по JSON-экспорту.

Ниже приведены типовые требования к данным и сценарии работы, реализованные в проекте.

Поддержка приёма произвольных бинарных кадров телеметрии по протоколу TCP.

Минимальные требования к развёртыванию и настройке системы.

1. Хранение кадров в оперативной памяти с ограничением по объёму.
2. Трансляция данных веб-клиенту по протоколу WebSocket.
3. Просмотр телеметрии в учебном режиме (таблица, карта, ориентация, температура).
4. Просмотр телеметрии в инженерном режиме (полный кадр, группы параметров, графики).
5. Выгрузка отчётов и архивов в форматах PDF, CSV и JSON.
6. Воспроизведение сеанса из JSON-файла с управлением скоростью и перемоткой.

На сегодняшний день существует ряд программных решений, предназначенных для сбора, передачи и обработки телеметрических данных, однако универсальные клиент-серверные приложения, ориентированные на оперативный приём телеметрии от внешних источников по протоколу TCP с последующей наглядной визуализацией и минимальными требованиями к настройке, представлены в недостаточном объёме. Многие существующие системы либо избыточно сложны, либо ориентированы на узкоспециализированные аппаратные комплексы, что затрудняет их использование в учебных и экспериментальных проектах.

Prometheus является широко распространённой системой мониторинга и сбора метрик. Данное решение ориентировано на работу с временными рядами и требует использования специализированного формата экспорта данных, а также дополнительной настройки компонентов сбора и визуализации. Использование Prometheus для приёма произвольных бинарных

телеметрических данных по протоколу TSP требует значительной доработки и усложняет архитектуру системы. Кроме того, настройка и сопровождение системы может быть избыточной для задач оперативного анализа телеметрии в рамках курсового проекта.



*Рисунок 1.1. Система мониторинга «Prometheus»<sup>1</sup>*

Система Zabbix представляет собой комплексное решение для мониторинга инфраструктуры и сетевых устройств. Она обладает широким функционалом, включающим оповещения, шаблоны и распределённую архитектуру, однако ориентирована преимущественно на мониторинг серверов и сетевого оборудования. Интерфейс системы содержит большое количество настроек и уровней конфигурации, что усложняет использование при необходимости быстрого приёма и отображения телеметрических данных от пользовательских источников. Кроме того, внедрение Zabbix требует использования базы данных и дополнительных сервисов, что увеличивает сложность развёртывания и эксплуатации.

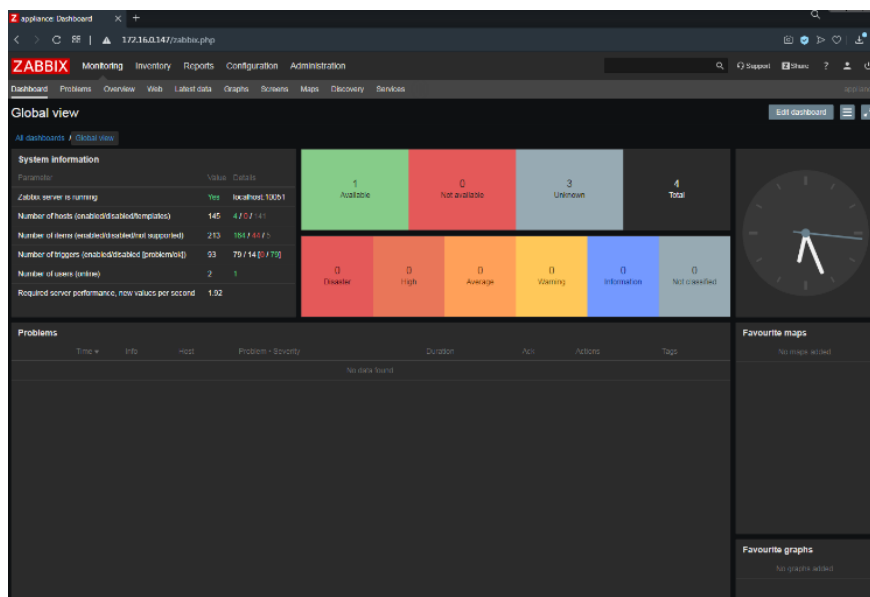


Рисунок 1.2. Система мониторинга «Zabbix»2

Таким образом, существующие решения либо обладают избыточным функционалом, либо не ориентированы на приём и обработку телеметрических данных в простом клиент-серверном приложении с использованием протокола TCP. В связи с этим разработка специализированного клиент-серверного приложения, обеспечивающего приём, обработку и визуализацию телеметрии с минимальными накладными расходами, является актуальной задачей.

### 1.3. Выбор средств разработки и технологий для клиентской части

#### Выбор инструментов:

Для разработки веб-интерфейса применяются три основных языка: HTML, CSS и JavaScript. Эти технологии лежат в основе любого браузерного приложения и обеспечивают разделение структуры, оформления и поведения, а в совокупности с подключаемыми библиотеками позволяют решать задачи визуализации телеметрии и взаимодействия с серверной частью. [4].

HTML — язык гипертекстовой разметки, описывающий структуру веб-страниц приложения: таблицы параметров телеметрии, элементы управления, контейнеры для графиков и карты, а также шапку с переключателями языка и темы оформления. [14].

**Преимущества:**

- Лёгкость в освоении и применении.
- Совместимость со всеми современными браузерами и платформами.
- Чёткое разделение структуры и оформления контента.
- Высокая скорость загрузки страниц.
- Возможность интеграции с JavaScript и сторонними библиотеками.

**Недостатки:**

- Недостаточный уровень интерактивности без подключения дополнительных технологий.
- Ограниченные возможности визуального оформления.
- Отображение может отличаться в зависимости от браузера.

CSS — каскадные таблицы стилей, отвечающие за внешний вид интерфейса: цветовые схемы, типографику, сетки, размеры элементов и переключение светлой и тёмной темы оформления.

**Преимущества:**

- Существенно сокращает время на оформление за счёт повторного использования стилей.
- Обеспечивает широкий спектр возможностей для дизайна и адаптации интерфейса.
- Улучшает скорость загрузки за счёт отделения стилей от контента.
- Адаптируется под различные экраны и устройства.
- Позволяет гибко управлять расположением элементов.

**Недостатки:**

- Возможны различия в отображении стилей в разных браузерах.
- Сложность в управлении при большом количестве вложенных правил.
- Потенциальная уязвимость при неправильной реализации или подключении внешних файлов.

JavaScript — язык программирования, обеспечивающий логику работы клиентской части: получение телеметрии по WebSocket, обновление таблиц и

графиков в реальном времени, переключение режимов просмотра, языка интерфейса и темы оформления, а также обработку запросов на экспорт отчётов. [30].

#### **Преимущества:**

- Ключевой инструмент для создания интерактивных клиент-серверных приложений.
- Отличное взаимодействие с HTML и CSS, что позволяет реализовать сложные интерфейсы.
- Высокая скорость выполнения в современных браузерах.
- Хорошая производительность при работе с динамическими данными.
- Обширная экосистема библиотек и инструментов, поддерживающих разработку.

#### **Недостатки:**

- Не имеет встроенных средств строгой типизации, что повышает риск ошибок.
- Доступ к серверным ресурсам ограничен возможностями браузера и протоколом обмена.
- Поведение может отличаться в разных средах исполнения и версиях браузеров.

Помимо базовых веб-технологий, в клиентской части применяются специализированные JavaScript-библиотеки, расширяющие возможности интерфейса в задачах визуализации телеметрии в реальном времени. В рамках данного проекта были выбраны Chart.js, MapLibre GL и встроенный в браузер WebSocket API.

Chart.js — JavaScript-библиотека для построения интерактивных графиков. В приложении Chart.js используется для отображения временных рядов параметров телеметрии: температурных значений и характеристик электропитания. Графики обновляются по мере поступления новых кадров и поддерживают переключение между текущим сеансом и режимом воспроизведения сохранённой записи.

MapLibre GL — открытая JavaScript-библиотека для отображения векторных карт в браузере. В приложении она применяется для отображения текущего положения аппарата и трека движения на карте по координатам, поступающим в потоке телеметрии в режиме реального времени.

Для приёма данных в реальном времени применяется встроенный в браузер WebSocket API: он устанавливает постоянное двустороннее соединение с сервером и получает кадры телеметрии в формате JSON по мере их декодирования.

#### **Преимущества использования выбранных библиотек:**

- Открытая лицензия и активная поддержка сообществом разработчиков.
- Низкие накладные расходы по сравнению с полноценными фреймворками.
- Адаптивность отображения под различные экраны и устройства.
- Совместимость с современными браузерами без дополнительных надстроек.
- Гибкие настройки внешнего вида графиков и карты. [12].

#### **Используемые технологии в клиентской части:**

- HTML — описывает структуру таблиц, элементов управления и контейнеров для графиков.
- CSS — отвечает за оформление, адаптивную вёрстку и переключение тем.
- JavaScript — реализует логику обновления данных и взаимодействия с сервером.
- Chart.js и MapLibre GL — формируют графики и отображают положение аппарата на карте.

Архитектура клиентской части построена на принципе разделения обязанностей: HTML отвечает за структуру, CSS — за оформление, JavaScript — за поведение и связь с сервером, а подключаемые библиотеки решают узкоспециализированные задачи визуализации. [5].



### **Выбор платформы:**

Для написания клиентского кода был выбран редактор кода Visual Studio Code от компании Microsoft.

Visual Studio Code представляет собой современный кроссплатформенный редактор исходного кода, предназначенный для разработки программного обеспечения различной сложности и направленности. Одной из ключевых характеристик данной платформы является обширная поддержка множества языков программирования, включая HTML, CSS, JavaScript, Go, Python и другие, что обеспечивает её универсальность в различных проектах. Visual Studio Code предлагает интуитивно понятный пользовательский интерфейс, а также мощный инструментарий для отладки, анализа кода и работы с системами контроля версий, что способствует повышению качества и надёжности программных решений. [25].

### **Преимущества:**

- Многофункциональность: включает обширный набор инструментов, автоматизирующих и оптимизирующих процессы разработки.
- Поддержка большого количества языков программирования, что расширяет спектр применимости среды.
- Возможности облачной интеграции, обеспечивающие развёртывание и сопровождение приложений в облачной инфраструктуре.
- Широкое профессиональное сообщество и официальная техническая поддержка от Microsoft.
- Высокая производительность и адаптивный интерфейс с возможностью персонализации.
- Гибкость за счёт подключения дополнительных расширений через специализированный маркетплейс.
- Встроенные средства контроля версий и командной разработки с использованием Git. [20].
- Кроссплатформенность: работа в операционных системах Windows, macOS и Linux.

**Недостатки:**

- В некоторых конфигурациях доступна ограниченная функциональность по сравнению с другими профессиональными IDE.
- Требовательность к системным ресурсам: при работе с большим количеством расширений возможны задержки в запуске и снижение скорости работы.

**Хранение данных на стороне клиента:**

В разрабатываемой системе для приёма и визуализации телеметрии отдельная клиентская база данных не используется. Все принятые и накопленные данные хранятся на стороне сервера в оперативном in-методе хранилище, а клиент работает с актуальным набором кадров и метаданных, получаемых по WebSocket и REST.

Такой подход исключает необходимость синхронизации между клиентом и сервером, упрощает обновление интерфейса и обеспечивает единый источник истины. При перезагрузке страницы клиент повторно запрашивает текущее состояние сеанса с сервера, а для оффлайн-анализа доступна выгрузка сеанса в JSON-файл и его последующая загрузка в режиме воспроизведения.

**Основные преимущества выбранной модели хранения данных:**

- Отсутствие зависимости от локальных средств хранения браузера и связанных с ними ограничений.
- Согласованное состояние между несколькими подключёнными клиентами в рамках одного сеанса.
- Простая модель доступа: текущее значение и история кадров получаются по WebSocket и REST.
- Возможность воспроизведения сохранённого сеанса локально, без обращения к серверу.

## 1.4. Выбор средств разработки и технологий для серверной части

### Выбор инструментов:

Для разработки серверной части приложения были выбраны следующие инструменты и платформы: язык программирования Go в качестве основного, стандартный пакет `net/http` для построения HTTP-сервера, библиотека `Gorilla WebSocket` для двусторонней связи с веб-клиентами, пакет `encoding/binary` для разбора двоичных кадров телеметрии и библиотека `gofpdf` для формирования PDF-отчётов. Выбор этих инструментов обусловлен их зрелостью, активной поддержкой сообществом и тем, что они входят в стандартную или широко распространённую экосистему Go. [1].

Go — компилируемый язык программирования общего назначения, разработанный компанией Google. Он сочетает простой синтаксис, статическую типизацию и встроенную поддержку конкурентного выполнения через горутин и каналы, что особенно важно при одновременном обслуживании TCP-приёмника, WebSocket-вещателя и HTTP-сервера. Выбор данной технологии обусловлен рядом рациональных и технически обоснованных факторов:

- Не накладывает жёстких ограничений на структуру проекта, позволяя разработчику самостоятельно определять необходимые компоненты, что удобно для прикладной серверной задачи.
- Встроенные средства параллелизма позволяют просто реализовать одновременную обработку TCP-потока, рассылку по WebSocket и обслуживание HTTP-запросов в рамках одного процесса.
- Стандартная библиотека `net/http` уже содержит всё необходимое для построения HTTP-сервера и REST-эндпоинтов, что ускоряет процесс разработки.
- Статическая типизация и компиляция в единственный исполняемый файл упрощают развёртывание сервера и его сопровождение.

В проекте Go используется для приёма потока телеметрии по TCP, разбора двоичных кадров, хранения сеансов и кадров в оперативной памяти, а также для

обмена данными с веб-клиентом через WebSocket и REST. [8].

Gorilla WebSocket — это библиотека для языка Go, обеспечивающая полноценную поддержку протокола WebSocket на стороне сервера. Она используется для установления постоянного двустороннего соединения с веб-клиентами и рассылки декодированных кадров телеметрии в режиме реального времени. Применение данной библиотеки обусловлено рядом функциональных и архитектурных преимуществ:

- Gorilla WebSocket предоставляет высокоуровневый API над протоколом WebSocket, что упрощает реализацию серверного хаба и обработки подключений клиентов.
- Поддерживает корректную обработку времени ожидания, контрольных кадров (ping/pong) и аккуратное закрытие соединений.
- Обеспечивает потоковую рассылку сообщений с минимальными накладными расходами, что критично для трансляции телеметрии в реальном времени.
- Хорошо сочетается со стандартным пакетом net/http и не требует дополнительных зависимостей или сторонних серверов. [7].

В проекте Gorilla WebSocket отвечает за реализацию веб-сокет-хаба: приём подключений от веб-клиентов по адресу /ws, рассылку каждого декодированного кадра подписчикам и аккуратное отключение клиентов при разрыве связи.

Пакеты encoding/binary и gofpdf играют вспомогательную, но важную роль. Первый используется для разбора входящих двоичных кадров телеметрии в порядке Little-Endian с извлечением всех полей структуры, а второй — для формирования PDF-отчётов по сеансам связи и кадрам телеметрии. Применение этих библиотек обусловлено их стабильностью и тем, что они входят в стандартную или широко распространённую экосистему Go:

- Пакет encoding/binary входит в стандартную библиотеку Go и не требует дополнительных зависимостей.
- Позволяет последовательно считывать поля бинарного кадра в указанном порядке байтов с минимальными накладными расходами.

- Библиотека `gofpdf` предоставляет программный интерфейс для построения PDF-документов на стороне сервера без внешних утилит.
- Поддерживает табличный вывод, настройку шрифтов и подходит для формирования отчётов по сеансам связи и кадрам телеметрии.
- Обе библиотеки распространяются по открытой лицензии и активно сопровождаются сообществом разработчиков.

### **Выбор платформы:**

В качестве среды разработки для серверного кода использовался редактор Visual Studio Code от компании Microsoft с официальным расширением Go, поддерживаемым командой Go-сообщества. [10]. Его выбор был обусловлен следующими преимуществами:

- Поддержка инструментов экосистемы Go (`gopls`, `go test`, `go vet`, отладчика Delve) непосредственно в редакторе через расширение.
- Наличие встроенного отладчика с пошаговым выполнением и точками останова, что упрощает выявление и устранение ошибок в серверной логике.
- Интеграция с системами контроля версий, такими как Git, обеспечивает возможность полноценного управления версиями, отслеживания изменений и совместной разработки.

Visual Studio Code применялся для написания, отладки и тестирования серверного кода. Использование единой среды разработки для клиентской и серверной частей сократило время переключения между компонентами проекта и упростило его сопровождение.

### **Выводы:**

В результате анализа предметной области и обзора существующих решений были определены функциональные требования к разрабатываемому клиент-серверному приложению для приёма и визуализации телеметрии. Проведено сравнение аналогов, на основе которого обоснован выбор ключевых функций: приём потока по TCP, декодирование двоичных кадров, рассылка по WebSocket в реальном времени, экспорт отчётов в форматах PDF, CSV и JSON,

а также режим воспроизведения сохранённых сеансов связи.

Также был осуществлён обоснованный выбор технологий: HTML, CSS и JavaScript с библиотеками Chart.js и MapLibre GL — для веб-клиента, язык Go со стандартной библиотекой net/http и Gorilla WebSocket — для серверной части, а также in-memory хранилище сеансов и кадров для оперативного доступа без накладных расходов на внешнюю СУБД. Предложенная архитектура ориентирована на низкую задержку, простоту сопровождения и удобство развёртывания, что позволяет эффективно решать задачи приёма и визуализации телеметрии летательных аппаратов.

## 2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

### 2.1. Проектирование функциональной модели

Разрабатываемая система состоит из следующих компонентов:

1. ТСР-приёмник телеметрии — принимает поток бинарных кадров от внешнего источника на порту 9000, формирует сеансы связи (одно ТСР-подключение — один сеанс). Состав хранимых данных:

- кадры телеметрии (полный набор параметров);
- информация о сеансах связи (идентификатор, адрес, временные метки);
- упрощённые параметры (для учебного режима);
- агрегаты для графиков (температуры, питание);
- журнал ключевых событий приёма.

2. Серверная часть — реализована на языке Go и представляет собой HTTP-сервер, объединяющий приёмник, хранилище и API-эндпоинты для выгрузок. Функции серверной части:

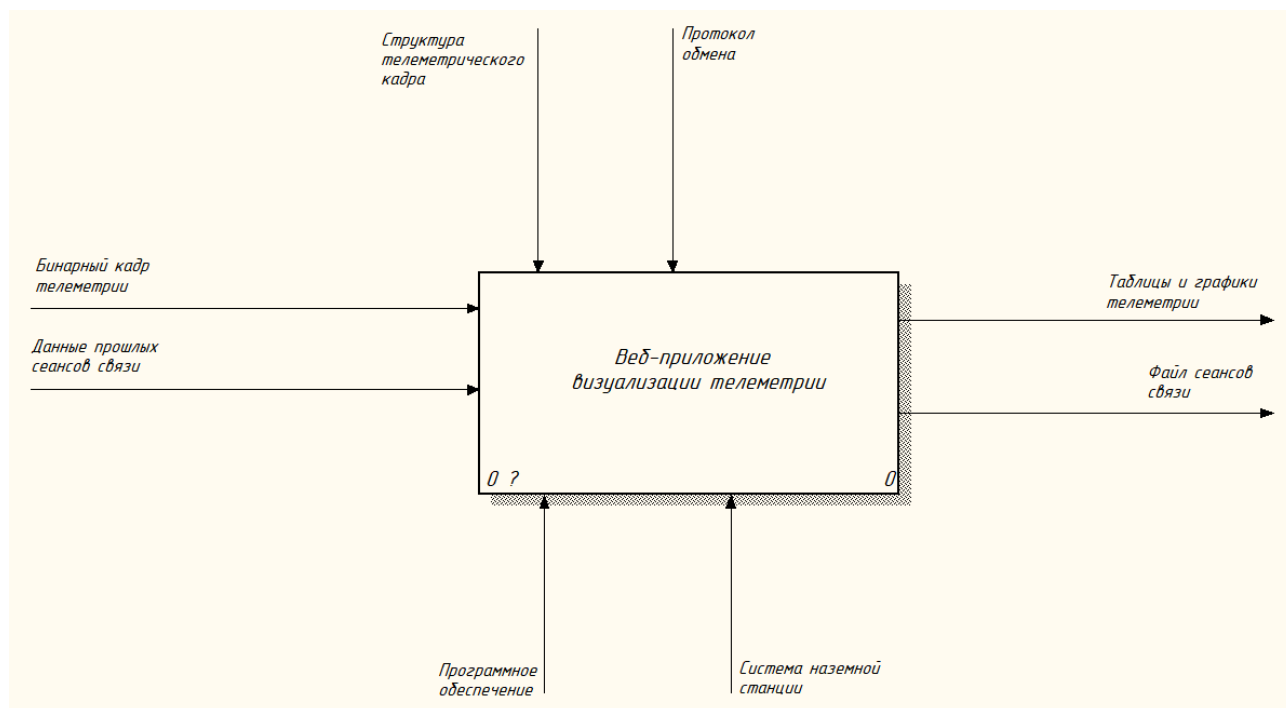
- приём и разбор кадров телеметрии;
- хранение сеансов и кадров в оперативной памяти;
- трансляция кадров клиентам по WebSocket;
- формирование отчётов и экспорт данных (PDF, CSV, JSON).

3. Клиентская часть — веб-приложение (HTML/CSS/JS), отдаваемое сервером по протоколу HTTP. Предоставляет три режима просмотра: учебный, инженерный и воспроизведение записанного сеанса. Поддерживает локализацию (RU/EN) и переключение тем оформления. [15].

Для обеспечения логической целостности и последовательной реализации функций приложения была разработана функциональная модель, отражающая ключевые процессы и взаимодействия между компонентами системы. В проектировании была выбрана методология IDEF0, которая позволяет наглядно

представить структуру и информационные потоки внутри разрабатываемого программного продукта. [6].

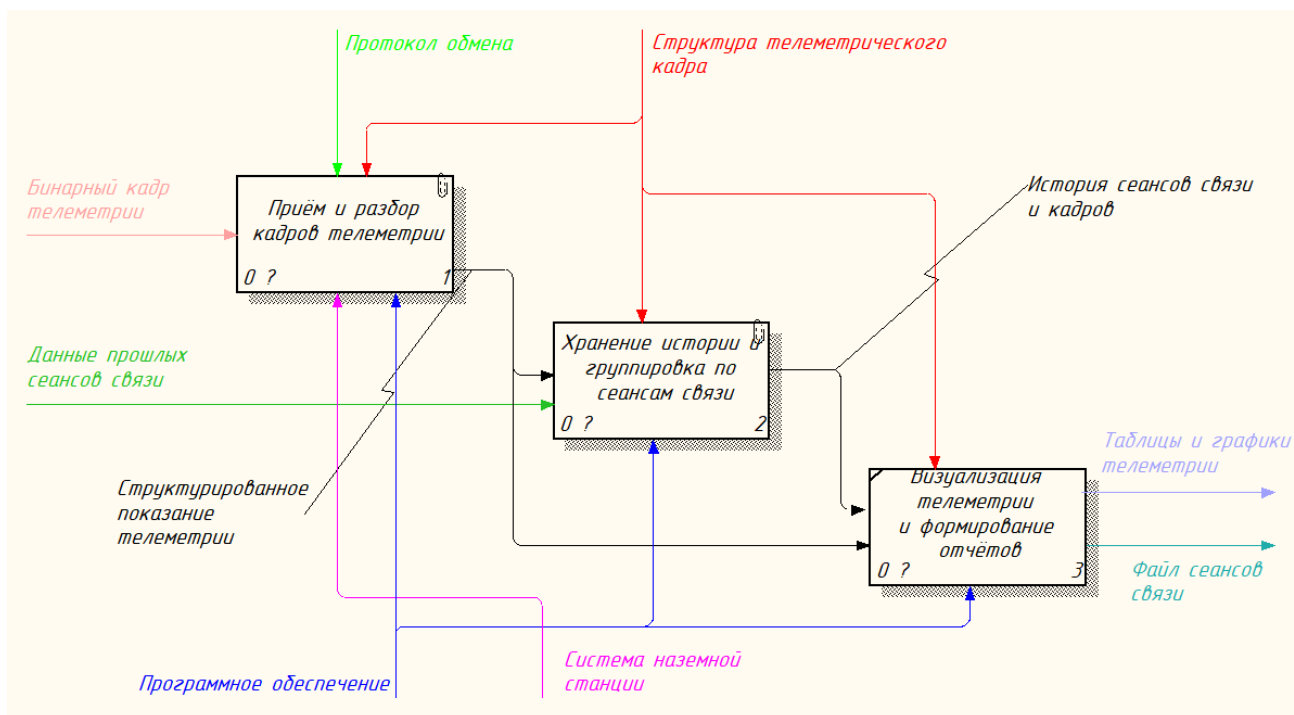
На первом уровне представлена общая функциональность приложения для приёма и визуализации телеметрии как единого блока, включающего входные данные (бинарный поток TCP), управляющие воздействия, используемые механизмы и выходные результаты (визуализация и отчёты). Данная диаграмма изображена на рисунке 2.



**Рис.2. Контекстная диаграмма приложения для приёма телеметрии**

На втором уровне выполнена декомпозиция основной функции на подпроцессы: приём по TCP, разбор кадров, хранение сеансов в памяти, трансляция данных по WebSocket и формирование экспортных отчётов. [13]. Данная диаграмма изображена на рисунке 3.





**Рис.3. Диаграмма декомпозиции**

Таким образом, представленные схемы позволяют наглядно отразить структуру и логику функционирования приложения для приёма и обработки телеметрии, а также обеспечить понимание ключевых процессов и их взаимосвязей в рамках проектируемой системы.

## 2.2. Принцип работы приложения

Разрабатываемое приложение функционирует как клиент-серверный сервис для приёма телеметрических данных от внешнего источника и их визуализации в веб-интерфейсе. Работа приложения начинается с запуска сервера, после чего он принимает входящие TCP-подключения и обслуживает запросы веб-клиентов.

Со стороны серверной части реализованы следующие ключевые функции:

- приём входящих TCP-подключений на порт 9000 как новых сеансов связи;
- разбор каждого входящего кадра в структуру телеметрии;
- сохранение кадров в памяти в рамках текущего сеанса связи;
- трансляция данных подключённым веб-клиентам через WebSocket;

- обработка REST-запросов на выгрузку отчётов и архивов сеансов;
- отдача статических файлов веб-интерфейса по HTTP.

Со стороны веб-клиента доступны следующие сценарии работы:

- просмотр телеметрии в реальном времени в выбранном режиме (учебном или инженерном);
- переключение между учебным и инженерным представлением одним нажатием;
- загрузка JSON-файла сеанса для воспроизведения с настройкой скорости и перемоткой;
- запрос экспорта отчётов и архивов (PDF/CSV/JSON) через кнопки в шапке.

Независимо от текущего режима просмотра пользователь может в любой момент переключиться между вкладками или прервать сеанс воспроизведения.

Для более детального разбора работы приложения была спроектирована схема клиент-серверного взаимодействия, представляющая обмен между тремя основными компонентами: внешним источником телеметрии (TCP-передатчиком), серверной частью на языке Go и веб-клиентом в браузере. Внешний источник подключается к серверу по TCP и передаёт бинарный поток кадров; серверная часть декодирует кадры, помещает их в оперативное in-memory хранилище и одновременно публикует подключённым WebSocket-клиентам; веб-клиент отображает данные пользователю и предоставляет интерфейс для запроса выгрузок и переключения режимов просмотра. [26]. Схема приведена на рисунке 4.

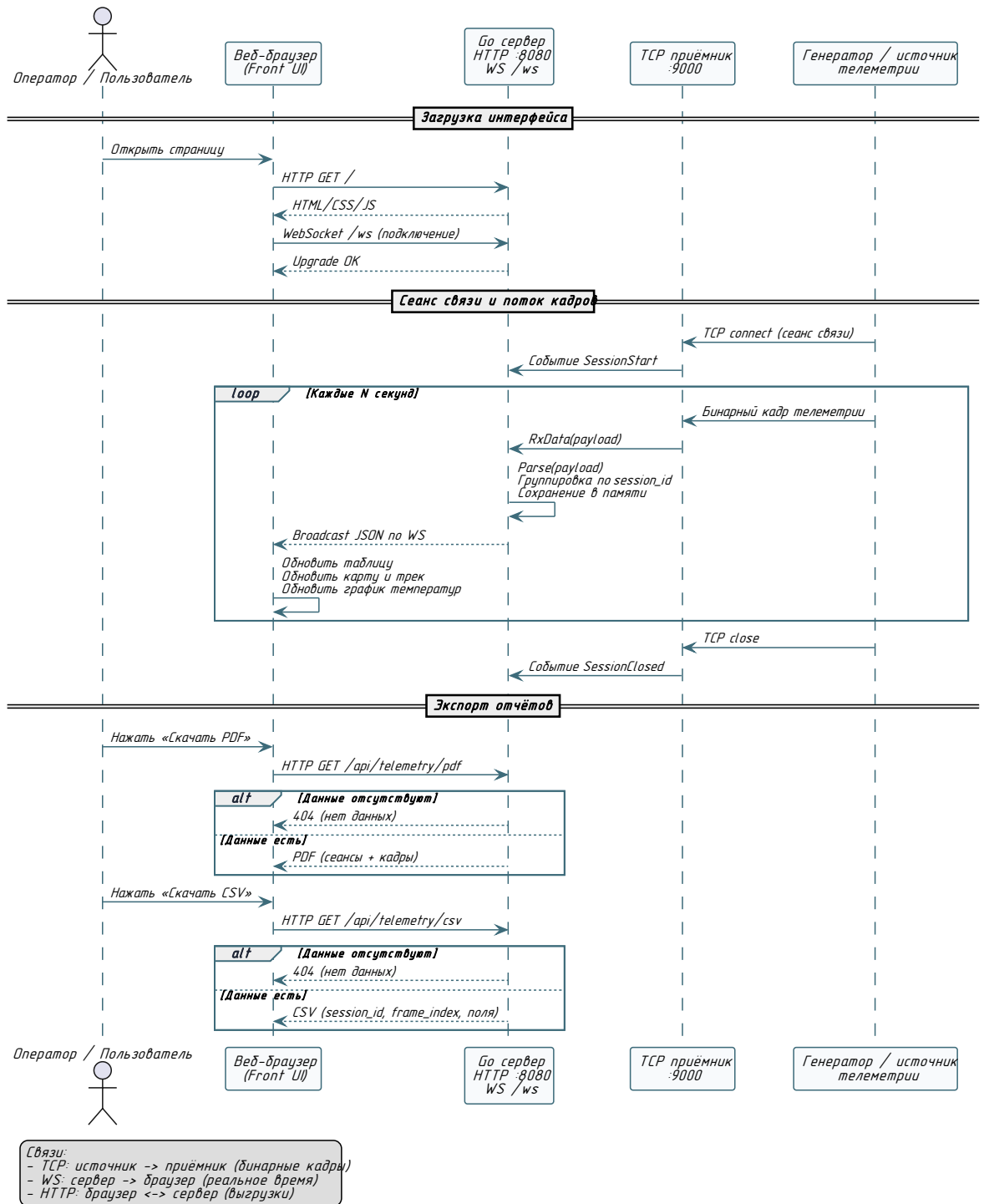


Рис.4. Схема клиент-серверного взаимодействия

Сервер является центральным компонентом системы и выполняет функции приёма данных, их обработки и координации взаимодействия между внешним источником и веб-клиентами. [27]. Он работает по следующему принципу:

1. Приём TCP-подключений и разбор бинарных кадров:

- Сервер слушает TCP-порт 9000 и принимает подключения от внешнего источника телеметрии.
  - Каждое подключение регистрируется как новый сеанс связи и помечается уникальным идентификатором.
2. Обработка HTTP-запросов:
    - Запросы от веб-клиента обслуживаются стандартным HTTP-маршрутизатором пакета net/http.
    - При обращении к /ws клиент подключается к WebSocket-хабу и начинает получать кадры в реальном времени.
  3. Рассылка данных клиентам:
    - Каждый декодированный кадр рассылается всем подключённым WebSocket-клиентам.
  4. Хранение сеансов и кадров:
    - Все принятые кадры сохраняются в оперативной памяти в виде структурированных сеансов.
    - Для одного сеанса хранится перечень кадров, его удалённый адрес и временные метки начала и завершения.
  5. Формирование и отправка ответа:
    - После запроса экспорта сервер собирает данные из памяти и формирует ответ в нужном формате.
    - При успешном формировании сервер возвращает файл с кодом состояния 200 OK.
    - При отсутствии данных или некорректном запросе возвращается соответствующий код ошибки и описание проблемы.
  6. Работа с реальным временем:
    - WebSocket-соединение позволяет мгновенно доставлять кадры клиенту без повторных запросов.
  7. Ограничения хранения:
    - Количество одновременно хранимых сеансов и кадров в одном сеансе ограничено заданными константами.

- При превышении лимита самые старые записи удаляются, освобождая память.

#### 8. Логирование:

- Все ключевые события сервера (подключение/отключение, ошибки разбора, ошибки записи) выводятся в стандартный лог.

Для трансляции телеметрии в режиме реального времени применяется протокол WebSocket, обеспечивающий двустороннюю связь между сервером и веб-клиентом. [9].

### 2.3. Структура данных

Структура данных приложения организована в виде взаимосвязанных в памяти структур, каждая из которых отвечает за хранение определённого типа информации, необходимой для работы системы:

1. Сеансы связи (CommunicationSession).
2. Кадры телеметрии (FullTelemetry — полный набор параметров).
3. События приёмника TCP (RxEvent).
4. Упрощённый кадр телеметрии (Telemetry).
5. Корневой объект экспорта сеанса в JSON.
6. Кадр в JSON-экспорте сеанса.
7. Параметры группы «Электропитание».
8. Параметры группы «АДС и датчики».
9. Параметры группы «Канал связи».
10. Параметры группы «Управление и статус».

На рисунке 5 представлена схема взаимодействия структур данных приложения.

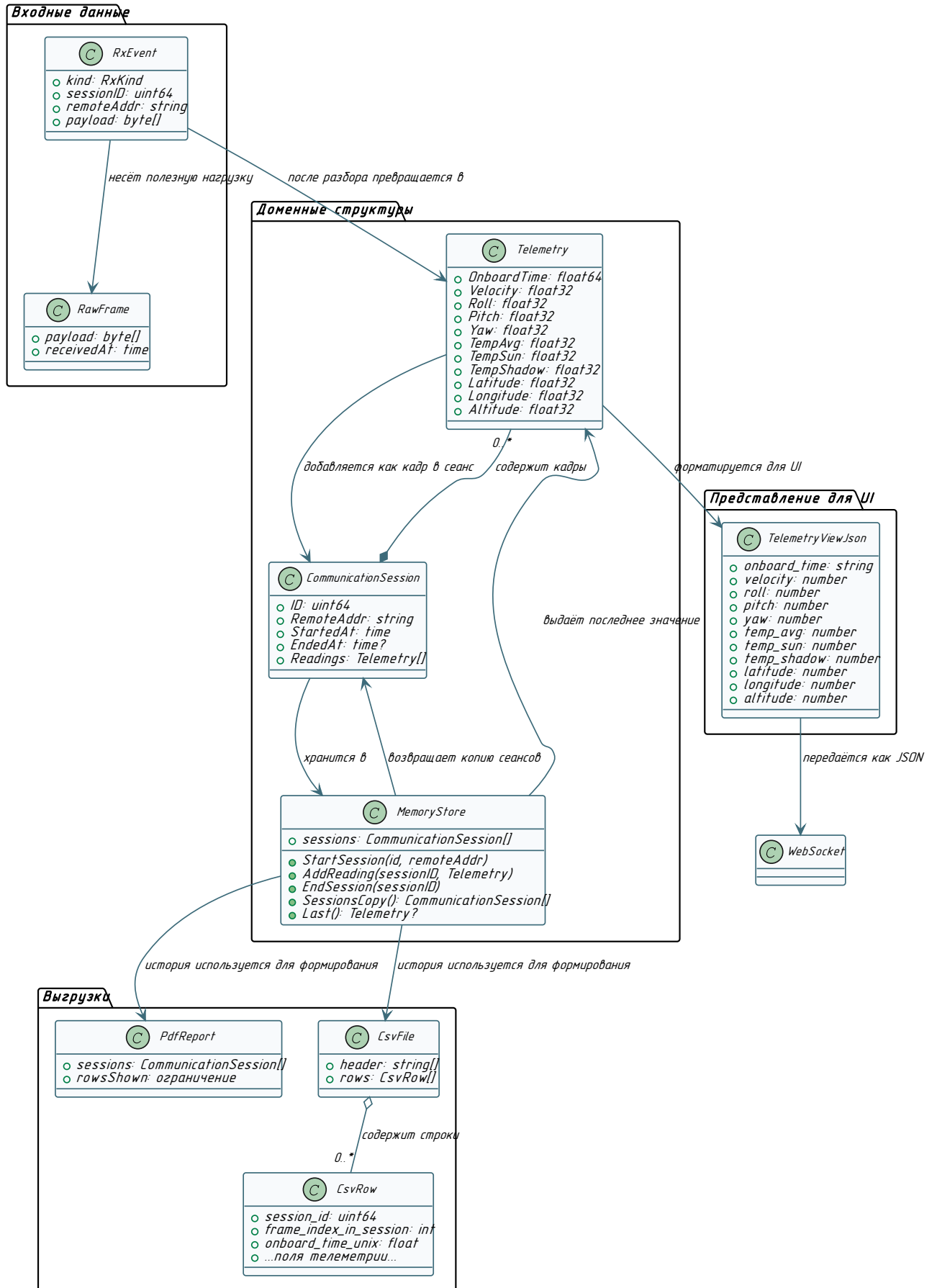


Рис.5. Схема взаимодействия структур данных приложения

Ниже приведено описание каждой из структур:

**Таблица 1 – Структура FullTelemetry (полный кадр телеметрии)**

| Наименование поля | Тип данных | Примечание                                  |
|-------------------|------------|---------------------------------------------|
| OnboardTime       | float64    | Бортовое время кадра, Unix-секунды          |
| PacketCounter     | uint32     | Счётчик пакетов от источника                |
| FrameType         | uint8      | Тип кадра ТМ                                |
| Mode              | uint8      | Режим аппарата (0 — safe, 1 — nom, 2 — sci) |
| CrcOk             | uint8      | Признак валидной CRC                        |
| Reserved          | uint8      | Резерв для выравнивания структуры           |
| Velocity          | float32    | Скорость, км/с                              |
| Roll              | float32    | Крен, °                                     |
| Pitch             | float32    | Тангаж, °                                   |
| Yaw               | float32    | Рысканье, °                                 |
| TempAvg           | float32    | Т средняя, °С                               |
| TempSun           | float32    | Т освещённой стороны, °С                    |
| TempShadow        | float32    | Т теневой стороны, °С                       |
| Latitude          | float32    | Широта, °                                   |
| Longitude         | float32    | Долгота, °                                  |
| Altitude          | float32    | Высота, км                                  |

|                |         |                                            |
|----------------|---------|--------------------------------------------|
| BatteryVoltage | float32 | Напряжение АКБ,<br>В                       |
| BatteryCurrent | float32 | Ток АКБ, мА                                |
| BatterySOC     | float32 | Заряд АКБ, %                               |
| SolarCurrent   | float32 | Ток СБ, мА                                 |
| SolarVoltage   | float32 | Напряжение СБ, В                           |
| GyroX          | float32 | Гироскоп по оси X,<br>°/с                  |
| GyroY          | float32 | Гироскоп по оси Y,<br>°/с                  |
| GyroZ          | float32 | Гироскоп по оси Z,<br>°/с                  |
| MagX           | float32 | Магнитометр по<br>оси X, мкТл              |
| MagY           | float32 | Магнитометр по<br>оси Y, мкТл              |
| MagZ           | float32 | Магнитометр по<br>оси Z, мкТл              |
| AccelX         | float32 | Акселерометр по<br>оси X, м/с <sup>2</sup> |
| AccelY         | float32 | Акселерометр по<br>оси Y, м/с <sup>2</sup> |
| AccelZ         | float32 | Акселерометр по<br>оси Z, м/с <sup>2</sup> |
| CpuTemp        | float32 | Температура<br>процессора ОВС,<br>°C       |
| RadioRSSI      | float32 | RSSI приёма, dBm                           |



|              |         |                           |
|--------------|---------|---------------------------|
| RadioTxPower | float32 | Мощность передатчика, dBm |
| Uptime       | uint32  | Время работы, с           |
| CommandCount | uint32  | Принято команд            |
| StatusFlags  | uint32  | Битовые флаги состояния   |

Описывает полный кадр телеметрии с максимальным набором параметров, принимаемый по TCP.

**Таблица 2 – Структура CommunicationSession (сеанс связи)**

| Наименование поля | Тип данных      | Примечание                              |
|-------------------|-----------------|-----------------------------------------|
| ID                | uint64          | Идентификатор сеанса связи              |
| RemoteAddr        | string          | Адрес удалённой стороны TCP             |
| StartedAt         | time.Time       | Время начала сеанса                     |
| EndedAt           | *time.Time      | Время завершения сеанса (nil — активен) |
| Readings          | []FullTelemetry | Принятые кадры телеметрии               |

Описывает сеанс связи как одно TCP-подключение и хранит ссылки на принятые кадры.

**Таблица 3 – Структура RxEvent (событие приёмника TCP)**

| Наименование поля | Тип данных | Примечание                          |
|-------------------|------------|-------------------------------------|
| Kind              | RxKind     | Тип события (start / data / closed) |
| SessionID         | uint64     | Идентификатор сеанса связи          |
| RemoteAddr        | string     | Адрес удалённой стороны TCP         |
| Payload           | []byte     | Бинарные данные кадра               |

Описывает события, передаваемые от приёмника ТСР к основному циклу обработки.

**Таблица 4 – Структура Telemetry (упрощённый кадр)**

| Наименование поля | Тип данных | Примечание                            |
|-------------------|------------|---------------------------------------|
| OnboardTime       | float64    | Бортовое время кадра,<br>Unix-секунды |
| Velocity          | float32    | Скорость, км/с                        |
| Roll              | float32    | Крен, °                               |
| Pitch             | float32    | Тангаж, °                             |
| Yaw               | float32    | Рысканье, °                           |
| TempAvg           | float32    | Т средняя, °С                         |
| TempSun           | float32    | Т освещённой<br>стороны, °С           |
| TempShadow        | float32    | Т теневой стороны, °С                 |
| Latitude          | float32    | Широта, °                             |
| Longitude         | float32    | Долгота, °                            |
| Altitude          | float32    | Высота, км                            |

Упрощённый набор параметров, используемый в учебном режиме веб-интерфейса.

**Таблица 5 – Структура экспорта сеанса в JSON (корневой объект)**

| Наименование поля | Тип данных                 | Примечание                                                |
|-------------------|----------------------------|-----------------------------------------------------------|
| Version           | int                        | Версия формата<br>экспорта                                |
| SessionID         | uint64                     | Идентификатор сеанса<br>связи                             |
| RemoteAddr        | string                     | Адрес удалённой<br>стороны ТСР                            |
| StartedAt         | string (RFC 3339)          | Время начала сеанса в<br>UTC                              |
| EndedAt           | string (RFC 3339,<br>опц.) | Время завершения<br>сеанса (отсутствует,<br>если активен) |

|        |       |                                            |
|--------|-------|--------------------------------------------|
| Frames | array | Массив кадров сеанса<br>(sessionFrameJSON) |
|--------|-------|--------------------------------------------|

Описывает корневой объект JSON-экспорта сеанса связи.

**Таблица 6 – Структура одного кадра в JSON-экспорте**

| Наименование поля | Тип данных | Примечание                                         |
|-------------------|------------|----------------------------------------------------|
| OnboardTimeUnix   | float64    | Бортовое время кадра<br>в Unix-секундах            |
| Simple            | object     | Упрощённый набор<br>полей кадра (учебный<br>режим) |
| Full              | object     | Полный набор полей<br>кадра (инженерный<br>режим)  |

Описывает один кадр в массиве frames JSON-экспорта сеанса.

**Таблица 7 – Параметры группы «Электропитание»**

| Наименование поля | Тип данных | Примечание           |
|-------------------|------------|----------------------|
| BatteryVoltage    | float32    | Напряжение АКБ,<br>В |
| BatteryCurrent    | float32    | Ток АКБ, мА          |
| BatterySOC        | float32    | Заряд АКБ, %         |
| SolarCurrent      | float32    | Ток СБ, мА           |
| SolarVoltage      | float32    | Напряжение СБ, В     |

Описывает параметры, относящиеся к группе электропитания аппарата.

**Таблица 8 – Параметры группы «АДС и датчики»**

| Наименование поля | Тип данных | Примечание                |
|-------------------|------------|---------------------------|
| GyroX             | float32    | Гироскоп по оси X,<br>°/с |
| GyroY             | float32    | Гироскоп по оси Y,<br>°/с |
| GyroZ             | float32    | Гироскоп по оси Z,<br>°/с |

|         |         |                                            |
|---------|---------|--------------------------------------------|
| MagX    | float32 | Магнитометр по<br>оси X, мкТл              |
| MagY    | float32 | Магнитометр по<br>оси Y, мкТл              |
| MagZ    | float32 | Магнитометр по<br>оси Z, мкТл              |
| AccelX  | float32 | Акселерометр по<br>оси X, м/с <sup>2</sup> |
| AccelY  | float32 | Акселерометр по<br>оси Y, м/с <sup>2</sup> |
| AccelZ  | float32 | Акселерометр по<br>оси Z, м/с <sup>2</sup> |
| CpuTemp | float32 | Температура<br>процессора ОВС,<br>°C       |

Описывает параметры, относящиеся к группе АДС и датчиков ориентации.

**Таблица 9 – Параметры группы «Канал связи»**

| Наименование поля | Тип данных | Примечание                   |
|-------------------|------------|------------------------------|
| RadioRSSI         | float32    | RSSI приёма, dBm             |
| RadioTxPower      | float32    | Мощность<br>передатчика, dBm |
| Uptime            | uint32     | Время работы, с              |

Описывает параметры, относящиеся к группе канала связи.

**Таблица 10 – Параметры группы «Управление и статус»**

| Наименование поля | Тип данных | Примечание                                        |
|-------------------|------------|---------------------------------------------------|
| PacketCounter     | uint32     | Счётчик пакетов от<br>источника                   |
| FrameType         | uint8      | Тип кадра ТМ                                      |
| Mode              | uint8      | Режим аппарата (0 —<br>safe, 1 — nom, 2 —<br>sci) |

|              |        |                                  |
|--------------|--------|----------------------------------|
| CrcOk        | uint8  | Признак валидной CRC             |
| CommandCount | uint32 | Принято команд за сеанс          |
| StatusFlags  | uint32 | Битовые флаги текущего состояния |

Описывает служебные параметры кадра и счётчики работы аппарата.

Структура данных приложения обеспечивает логичную организацию и хранение всех ключевых параметров телеметрии в оперативной памяти, а также удобное представление данных при выгрузке и в режиме воспроизведения. [23].

## 2.4. Алгоритм обработки телеметрии

Для обработки потока телеметрии в приложении используется набор связанных функций, реализованных на языке Go. [2]. Алгоритм работает в потоковом режиме: данные поступают от TCP-приёмника в виде набора событий, далее проходят разбор, фильтрацию и трансляцию подключённым веб-клиентам. [21].

В отличие от пакетной обработки, в системе организован непрерывный поток: каждый новый кадр сразу разбирается и публикуется клиентам, а также сохраняется в памяти для последующих выгрузок и режима воспроизведения. Это позволяет обеспечить минимальную задержку и одновременную поддержку нескольких подключений.

### Структура данных:

В алгоритме используются несколько ключевых структур, определяющих формат обмена и хранения:

- RxEvent — событие приёмника TCP (начало сеанса, кадр данных, завершение сеанса);
- FullTelemetry и Telemetry — структуры кадра телеметрии (полный и упрощённый наборы параметров);
- CommunicationSession и MemoryStore — описание сеанса связи и in-

memory хранилища.

В функции StartListener реализован цикл приёма входящих TCP-подключений; для каждого подключения формируется набор событий, которые отправляются в общий канал событий для последующей обработки. [22].

#### **Алгоритм определения сеанса:**

В обработчике событий приёмника реализована следующая логика классификации событий:

- RxSessionStart — начало нового сеанса связи: в хранилище создаётся новый объект CommunicationSession.
- RxData — поступление кадра данных: бинарный буфер передаётся в функцию разбора, после чего полученные структуры добавляются к текущему сеансу.
- RxSessionClosed — завершение сеанса: в объекте сеанса фиксируется время завершения.

#### **Алгоритм разбора кадра:**

Функция Parse/ParseAll принимает буфер бинарных данных и выполняет разбор:

- Считывание полей структуры в порядке Little-Endian с использованием стандартного пакета encoding/binary.
- Извлечение упрощённого представления Telemetry из полного кадра FullTelemetry для учебного режима.
- Возврат структурированных данных в основной цикл обработки для последующей рассылки и сохранения.

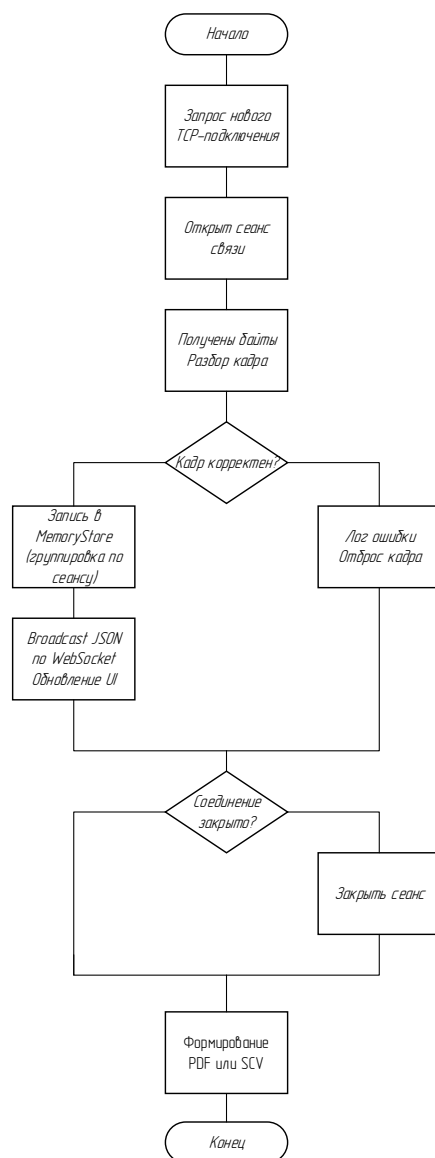
#### **Доставка кадра клиенту:**

После разбора кадра выполняется его упаковка в JSON и рассылка подключённым WebSocket-клиентам:

- Формирование JSON-сообщения с полями simple/full через функцию форматирования FormatWSMessage.
- Передача сообщения в Hub.Broadcast — функция рассылает данные всем подключённым клиентам.

- При ошибке записи в конкретного клиента он удаляется из списка активных подключений.
- Параллельно кадр сохраняется в MemoryStore для последующих выгрузок и режима воспроизведения.

Для более наглядного представления логики обработки телеметрии разработана блок-схема, отражающая последовательность основных этапов обработки данных и принятия решений внутри системы. Схема иллюстрирует, как поступающие по ТСР кадры классифицируются, разбираются, передаются веб-клиентам и сохраняются в памяти. [17]. Блоки схемы охватывают ключевые функциональные компоненты: приём, разбор, доставку, сохранение и формирование выгрузок. Данная схема изображена на рисунке 6.



**Рис.6. Блок-схема алгоритма обработки телеметрии**

## **Выводы:**

В ходе проектирования была разработана архитектура приложения для приёма и визуализации телеметрии, включающая серверную и клиентскую части, а также структурированное in-memory хранилище. Основное внимание уделено взаимодействию компонентов, обеспечивающему стабильную и низколатентную работу системы.

Клиентская часть представляет собой статическое веб-приложение на базе HTML, CSS и JavaScript, которое отдаётся сервером и работает в любом современном браузере. [11]. Она предоставляет три режима работы: учебный, инженерный и режим воспроизведения записанного сеанса.

Серверная часть реализована на языке Go и сочетает TCP-приёмник, обработчик кадров, WebSocket-хаб и HTTP-эндпоинты выгрузок. Архитектура построена на разделении модулей: receiver, processor, broadcaster, storage, api.

Доставка данных в реальном времени организована через Gorilla WebSocket. Для выгрузок реализованы эндпоинты с использованием стандартных пакетов и библиотеки gofpdf.

Хранение данных организовано в оперативной памяти посредством структуры MemoryStore, что упрощает развёртывание и не требует установки СУБД.

Алгоритм обработки кадров учитывает разбор бинарных данных, классификацию событий приёмника и одновременную доставку клиентам, что обеспечивает оперативность системы.

Таким образом, во второй главе был реализован полный цикл проектирования архитектурных решений, обеспечивших основу для дальнейшей разработки и внедрения приложения.



### 3. КОНТРОЛЬ КАЧЕСТВА И ИНТЕГРАЦИЯ КОМПОНЕНТОВ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

### 3.1. Разработка тестов и результаты тестирования

Тестирование разработанных модулей проводилось с целью проверки корректности их работы и выявления возможных ошибок. Особое внимание при тестировании уделялось разбору кадров телеметрии, экспорту отчётов и устойчивости работы сервера при множественных ТСР-подключениях. Результаты тестирования, а также описание найденных ошибок и тест-кейсов представлены в таблицах 11–12.

### Таблица 11 – Обзор ошибок в системе приёма и обработки телеметрии

| №<br>ошибк<br>и | Название модуля / функции            | Описание<br>ошибки                                                                                                      | Важност<br>ь | Ошибка<br>исправлен<br>а |
|-----------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------|--------------------------|
| 1               | processor.Parse()                    | При неполном<br>кадре парсер<br>возвращал<br>ошибку чтения<br>без проверки<br>размера буфера —<br>возможна паника       | средняя      | Да                       |
| 2               | broadcaster.Hub.Broadcast()          | При ошибке<br>записи в<br>WebSocket клиент<br>удалялся, но<br>соединение могло<br>остаться в карте<br>при гонке доступа | высокая      | Да                       |
| 3               | storage.MemoryStore.AddReading(<br>) | Отсутствовало<br>логирование<br>ситуации, когда<br>AddReading                                                           | низкая       | Да                       |

|   |                                  |                                                                     |         |     |
|---|----------------------------------|---------------------------------------------------------------------|---------|-----|
|   |                                  | вызывается для несуществующего сеанса                               |         |     |
| 4 | api.TelemetryPDFHandler()        | При пустом хранилище возвращался код 500 вместо 404                 | низкая  | Да  |
| 5 | tcp.handle()                     | Не закрывался TCP-приёмник при ошибке Read, отличной от EOF         | средняя | Нет |
| 6 | api.TelemetryFullLatestHandler() | При пустом хранилище ответ возвращался без корректного Content-Type | низкая  | Да  |

**Таблица 12 – Тест-кейсы**

| <b>Критерий теста</b>              | <b>№ теста</b> | <b>Входные данные</b>                | <b>Ожидаемый результат</b>                            | <b>Фактический результат</b>                          |
|------------------------------------|----------------|--------------------------------------|-------------------------------------------------------|-------------------------------------------------------|
| Приём корректного кадра            | 1              | Бинарный пакет полной длины          | Парсер возвращает заполненную структуру FullTelemetry | Парсер возвращает заполненную структуру FullTelemetry |
| Приём слишком короткого пакета     | 2              | Буфер длиной 16 байт                 | Ошибка разбора, кадр игнорируется                     | Ошибка разбора, кадр игнорируется                     |
| Несколько TCP-сеансов одновременно | 3              | 3 одновременных подключения на :9000 | Каждое подключение получает собственный session_id    | Каждое подключение получает собственный session_id    |

|                                 |    |                                           |                                                         |                                                         |
|---------------------------------|----|-------------------------------------------|---------------------------------------------------------|---------------------------------------------------------|
| Передача кадра по WebSocket     | 4  | Подключение к /ws и приём 5 кадров        | Клиент получает 5 JSON-сообщений в реальном времени     | Клиент получает 5 JSON-сообщений в реальном времени     |
| Запрос PDF при пустом хранилище | 5  | GET /api/telemetry/pdf без данных         | Ответ 404 с описанием                                   | Ответ 404 с описанием                                   |
| Запрос полного CSV              | 6  | GET /api/telemetry/csv/full               | CSV с заголовком из 39 колонок и данными по всем кадрам | CSV с заголовком из 39 колонок и данными по всем кадрам |
| Экспорт JSON всех сеансов       | 7  | GET /api/telemetry/sessions/export        | Массив сеансов с непустыми frames[]                     | Массив сеансов с непустыми frames[]                     |
| Воспроизведение JSON-сессии     | 8  | Загрузка экспортированного JSON в UI      | Воспроизведение кадров с заданной скоростью             | Воспроизведение кадров с заданной скоростью             |
| Превышение лимита сеансов       | 9  | Создание сеанса при заполненном хранилище | Самый старый сеанс удаляется из памяти                  | Самый старый сеанс удаляется из памяти                  |
| Переключение языка интерфейса   | 10 | Переключение RU↔EN в UI                   | Все метки и подсказки локализуются мгновенно            | Все метки и подсказки локализуются мгновенно            |

В результате тестирования были выявлены отдельные недочёты в логике работы функций сервера и компонентов приёма. Все критические ошибки были успешно исправлены. Проведённые тесты показали, что при корректных входных данных приложение стабильно принимает кадры, корректно их разбирает, передаёт веб-клиенту и формирует отчёты по запросу. Работа системы признана корректной. [16].

### 3.2. Интерфейс веб-приложения

Веб-интерфейс реализован как статический сайт, отдаваемый сервером по протоколу HTTP. Для отображения использованы стандартные веб-технологии — HTML, CSS и JavaScript, для построения графиков — библиотека Chart.js, для отображения карт — MapLibre GL JS. Подключение к источнику данных в реальном времени выполняется через WebSocket. [29].

Визуальная часть приложения построена на компонентном подходе на уровне отдельных JS-модулей (table.js, advanced-table.js, chart.js, map.js, orientation.js, replay.js и других), что обеспечило модульность, повторное использование элементов и упрощённую поддержку интерфейса.

Интерфейс предоставляет три режима работы: учебный просмотр, инженерный просмотр и воспроизведение записанного сеанса. Для удобства предусмотрены переключатели языка интерфейса (RU/EN) и темы оформления (светлая/тёмная), а также кнопки выгрузок отчётов и архивов в шапке страницы. [24].

На приведённых далее изображениях представлена визуальная структура интерфейса веб-приложения, отражающая основные элементы взаимодействия пользователя с системой и логическую организацию навигации между режимами.

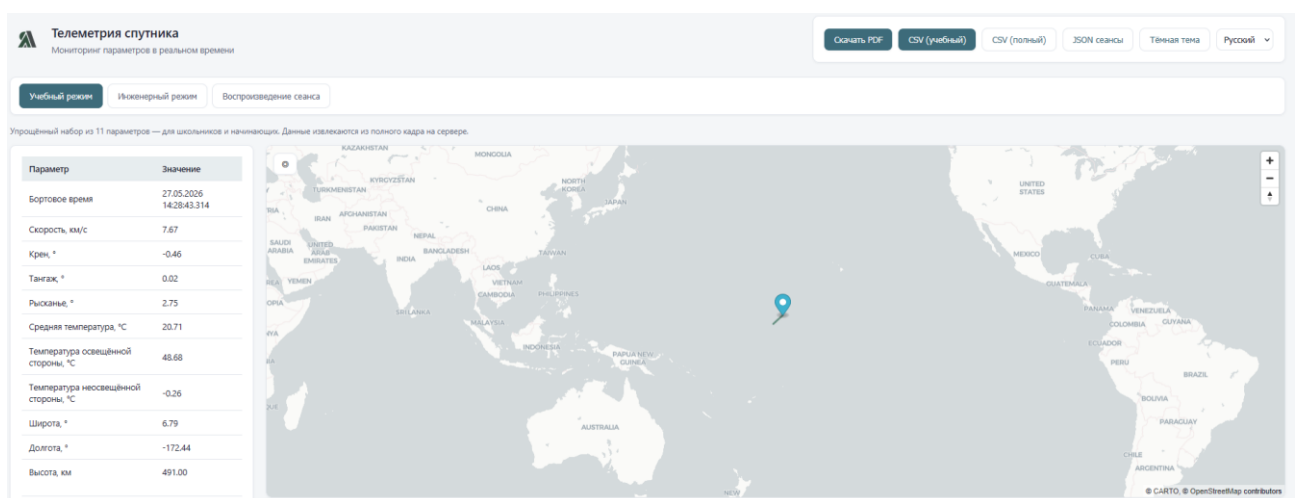
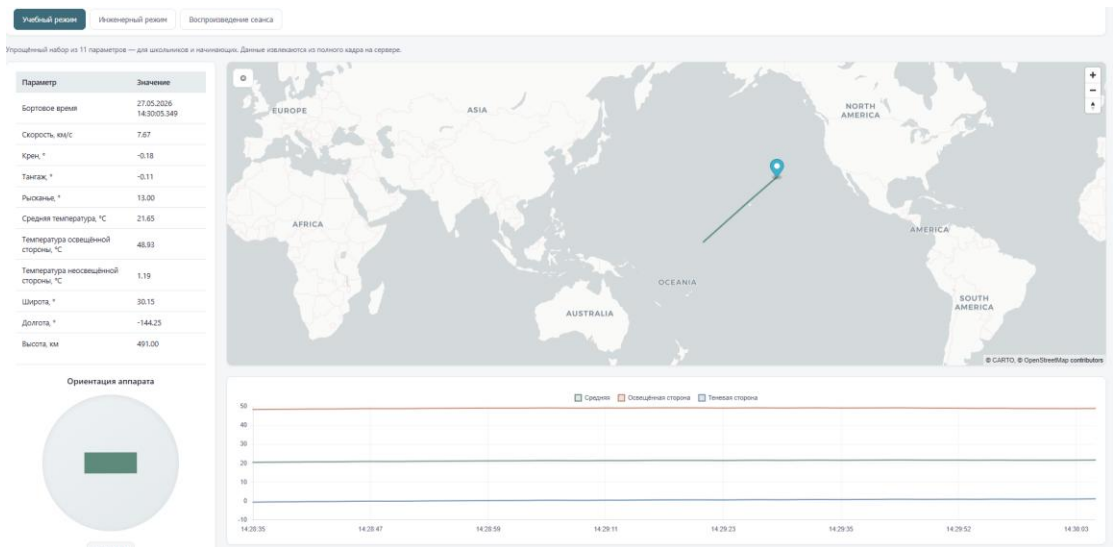


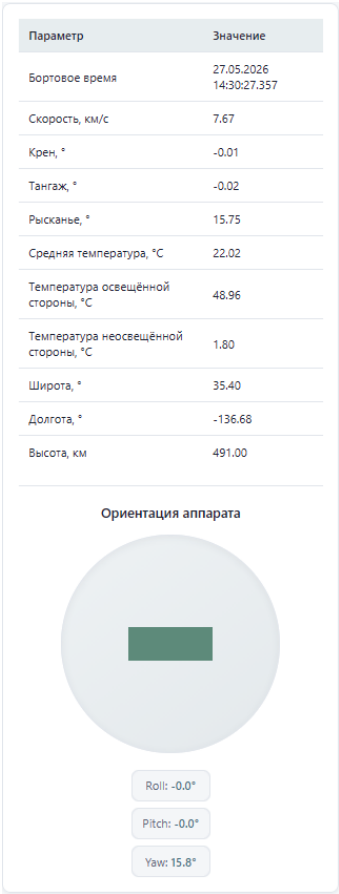
Рис.7. Главный экран веб-приложения

После открытия веб-приложения пользователю отображается главный экран с шапкой, кнопками выгрузок и переключателями языка и темы. В верхней навигационной панели доступны три вкладки: «Учебный режим», «Инженерный режим» и «Воспроизведение сеанса».

**Общие элементы интерфейса:**

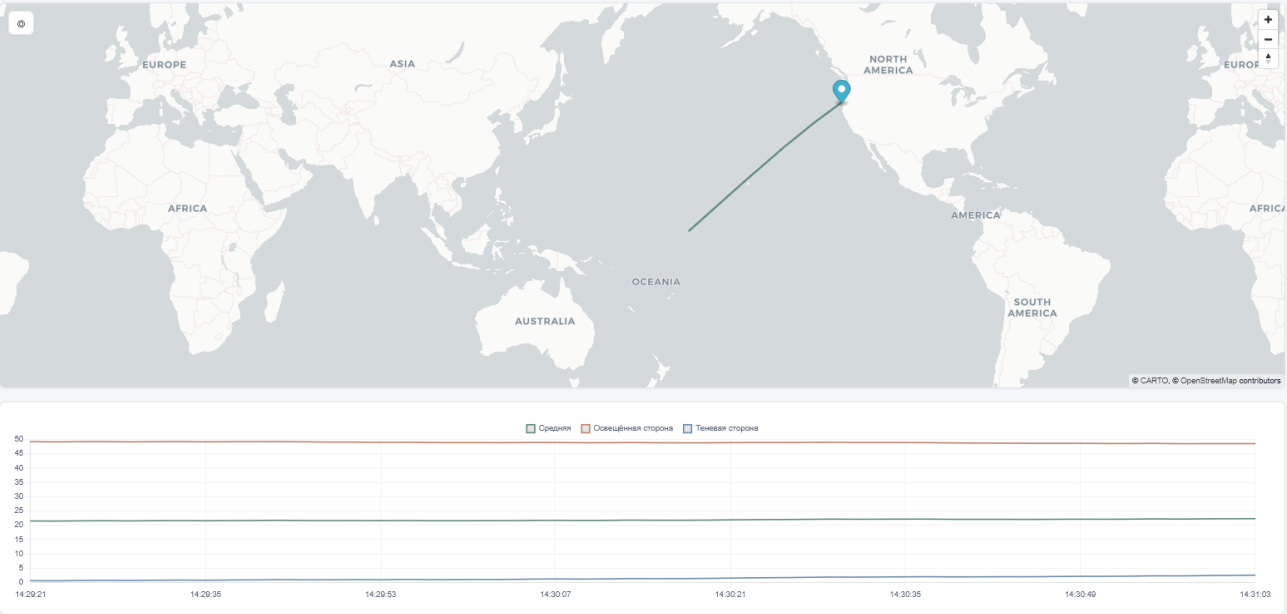


**Рис.8. Учебный режим**



**Рис.9.1. Таблица параметров и блок ориентации в учебном режиме**

В учебном режиме отображается упрощённая таблица из 11 параметров: бортовое время, скорость, ориентация (крен, тангаж, рысканье), три температуры, координаты и высота. Под таблицей расположен блок визуализации ориентации аппарата с численными значениями. Справа отображается карта положения и график температур.



**Рис.9.2. Карта положения и график температур**

Карта реализована на базе MapLibre GL JS и отображает текущее положение аппарата и пройденный трек. Цвет линии трека и подложка карты адаптируются к выбранной теме оформления. График температур показывает изменение средней, освещённой и теневой температур во времени, обновляясь по мере поступления новых кадров.

**Инженерный режим:**

| СЛУЖЕБНЫЕ ПОЛЯ КАДРА            |                         | ОРБИТА И ОРИЕНТАЦИЯ |          | ТЕРМОКОНТРОЛЬ        |          |
|---------------------------------|-------------------------|---------------------|----------|----------------------|----------|
| Параметр                        | Значение                | Параметр            | Значение | Параметр             | Значение |
| Бортовое время                  | 27.05.2026 14:31:27.379 | Скорость, км/с      | 7.67     | Т средняя, °C        | 22.12    |
| Счётчик пакетов                 | 93.00                   | Широта, °           | 48.31    | Т освещённая, °C     | 47.29    |
| Тип кадра TM                    | 1.00                    | Долгота, °          | -110.56  | Т тень, °C           | 3.25     |
| Режим KA (0=safe, 1=nom, 2=sci) | 2.00                    | Высота, км          | 491.00   | Т процессора ОВС, °C | 29.95    |
| CRC OK                          | 1.00                    | Крен, °             | 0.18     | ЭЛЕКТРОПИТАНИЕ (ОЭС) |          |
| Uptime, c                       | 186.00                  | Тангаж, °           | 0.10     |                      |          |
| Принято команд                  | 5.00                    | Рысканье, °         | 25.25    | Параметр             | Значение |
| Флаги состояния                 | 3.00                    | ТЕРМОКОНТРОЛЬ       |          | Напряжение АКБ, В    | 8.09     |
| ОРБИТА И ОРИЕНТАЦИЯ             |                         | Параметр            | Значение | Ток АКБ, мА          | -49.34   |
|                                 |                         | Параметр            | Значение | Заряд АКБ, %         | 89.20    |
| Скорость, км/с                  | 7.67                    | Т средняя, °C       | 22.05    | Ток СБ, мА           | 251.66   |
| Широта, °                       | 48.31                   | Т освещённая, °C    | 47.37    | Напряжение СБ, В     | 4.90     |

**Рис.10.1. Полный кадр телеметрии по группам параметров**

В инженерном режиме отображается полный кадр телеметрии (~35 параметров), сгруппированный по разделам: служебные поля кадра, орбита и ориентация, термоконтроль, электропитание, АДС и датчики, канал связи.

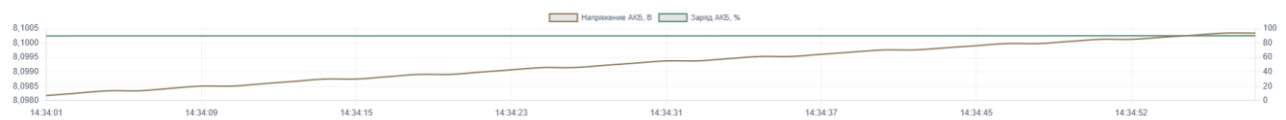


Рис.10.2. График электропитания (АКБ и СБ)

В инженерном режиме дополнительно отображается график параметров электропитания: напряжение АКБ и заряд АКБ по двум осям. Карта и блок ориентации работают аналогично учебному режиму, но с использованием значений из полного кадра.

| ЭЛЕКТРОПИТАНИЕ (ОЭС) |          | АДС И ДАТЧИКИ        |          |
|----------------------|----------|----------------------|----------|
| Параметр             | Значение | Параметр             | Значение |
| Напряжение АКБ, В    | 8.09     | Гироскоп X, °/с      | 1.10     |
| Ток АКБ, мА          | -50.21   | Гироскоп Y, °/с      | -0.81    |
| Заряд АКБ, %         | 89.24    | Гироскоп Z, °/с      | 0.15     |
| Ток СБ, мА           | 249.48   | Магнитометр X, μT    | 19.19    |
| Напряжение СБ, В     | 4.90     | Магнитометр Y, μT    | 10.68    |
| АДС И ДАТЧИКИ        |          | Магнитометр Z, μT    | 42.04    |
| Параметр             | Значение | Акселерометр X, м/с² | -0.01    |
| Гироскоп X, °/с      | 1.36     | Акселерометр Y, м/с² | -0.00    |
| Гироскоп Y, °/с      | -0.61    | Акселерометр Z, м/с² | -9.80    |
| Гироскоп Z, °/с      | 0.13     | КАНАЛ СВЯЗИ          |          |
| Магнитометр X, μT    | 19.95    | Параметр             | Значение |

Рис.10.3. Подробное представление параметров

Каждая группа параметров отображается отдельным блоком и обновляется в реальном времени по мере поступления новых кадров через WebSocket. При свернутой вкладке обновление данных не выполняется для экономии ресурсов браузера.

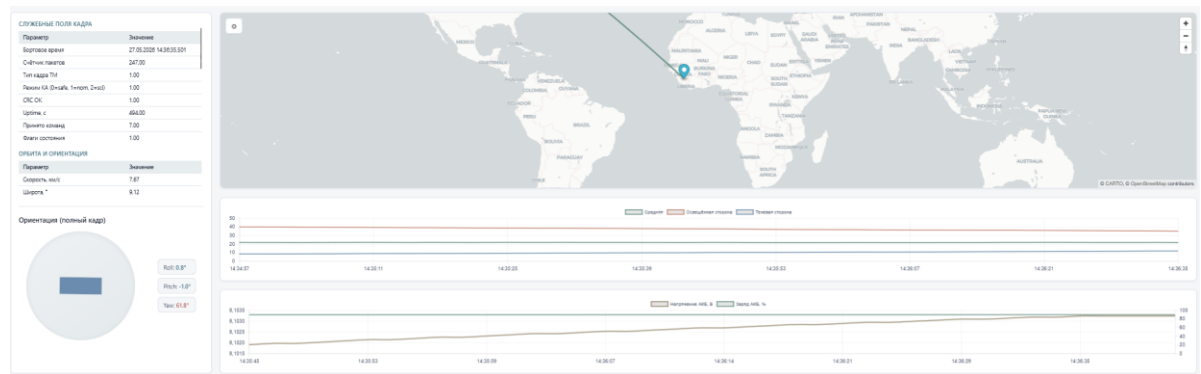
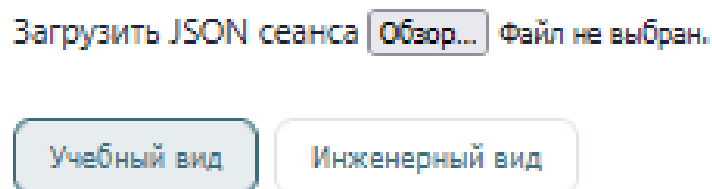


Рис.10.4. Резервное получение последнего кадра

Если WebSocket-канал временно недоступен, инженерный режим автоматически запрашивает последний полный кадр через REST-эндпоинт /api/telemetry/full/latest каждые 5 секунд, что повышает устойчивость отображения к кратковременным сбоям связи.

#### Воспроизведение сеанса:



**Рис.11.1. Загрузка JSON-сеанса**

В режиме воспроизведения пользователь загружает JSON-файл сеанса, экспортированный сервером, и просматривает кадры с возможностью полного контроля воспроизведения. Файл выбирается через стандартный диалог загрузки.



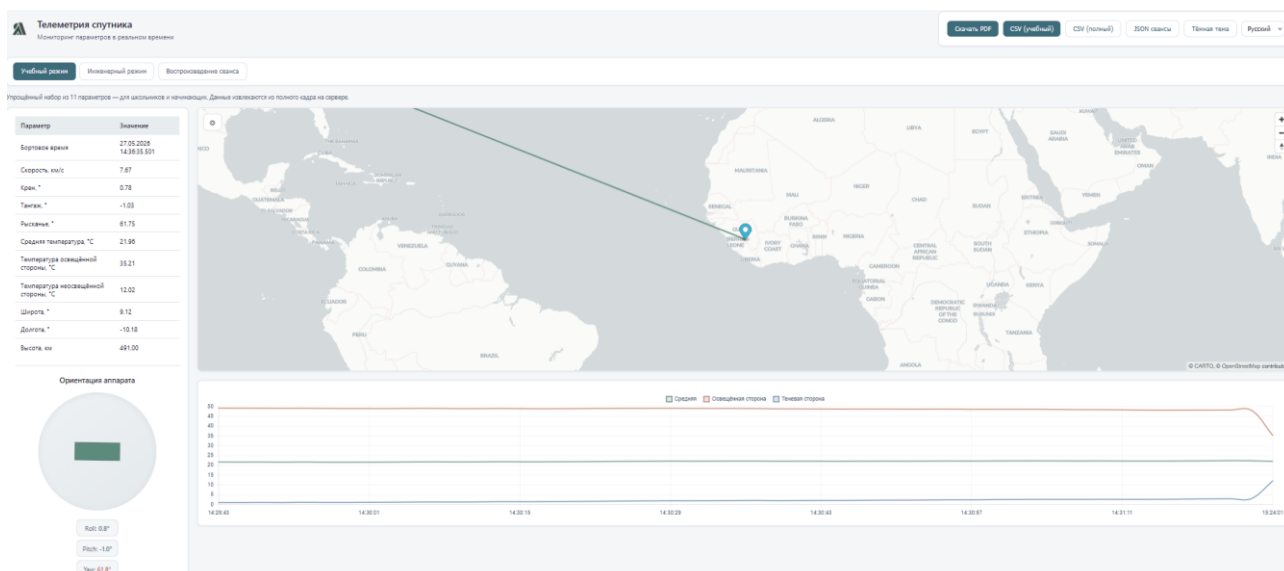
**Рис.11.2. Управление воспроизведением**

Доступны элементы управления: воспроизведение, остановка, перемотка по кадрам с помощью ползунка и выбор скорости воспроизведения ( $0,25\times$ – $8\times$ ). Также можно переключать представление между учебным и инженерным видом без перезагрузки сеанса.

### 3.3. Руководство пользователя

Для запуска веб-приложения необходимо открыть любой современный браузер и перейти по URL-адресу запущенного сервера. Если сервер запущен и принимает кадры телеметрии, на экране отобразится главная страница приложения. [19].

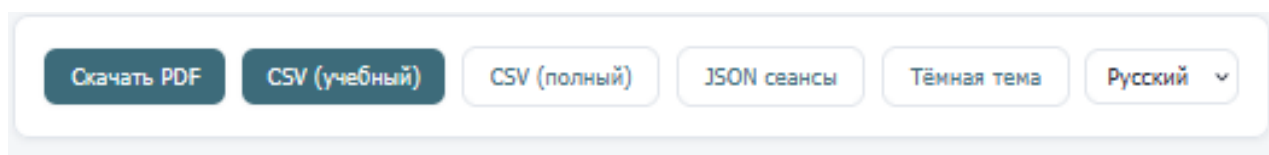




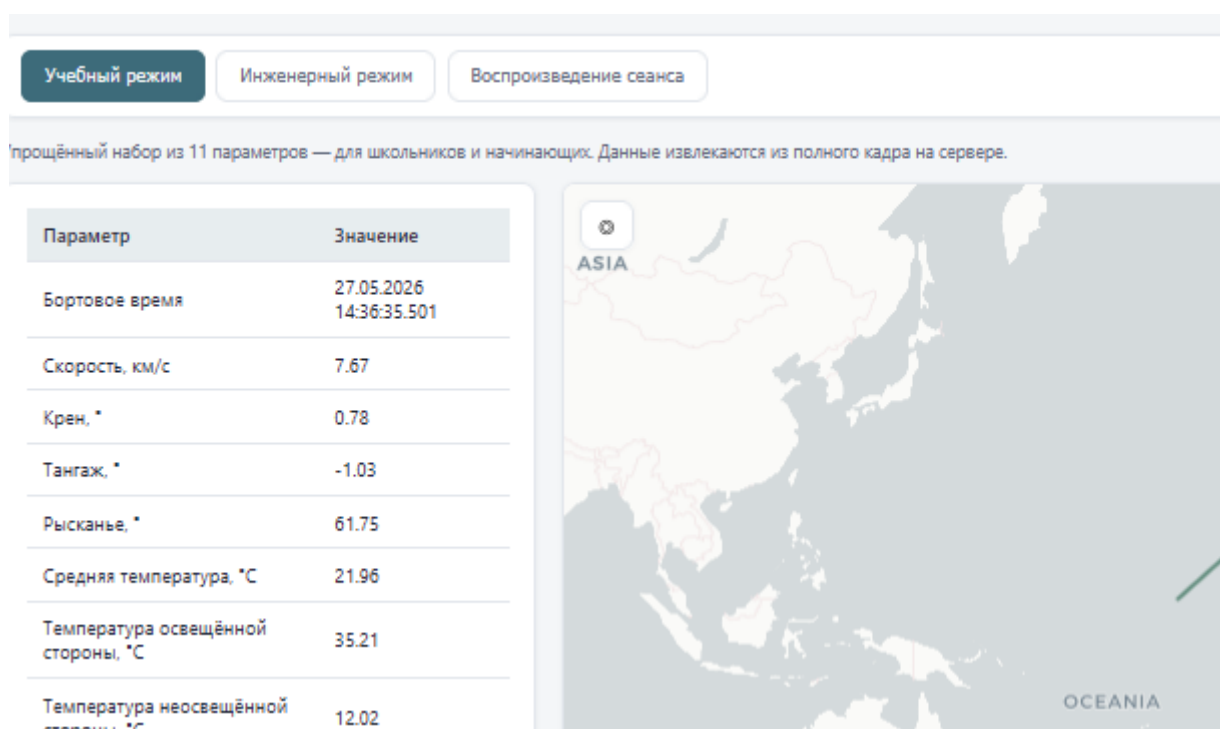
**Рис.12. Главная страница веб-приложения**

После открытия страницы доступны три режима просмотра. Для каждого режима в верхней панели предусмотрены кнопки выгрузок и переключатели языка и темы.

### Общие функции:

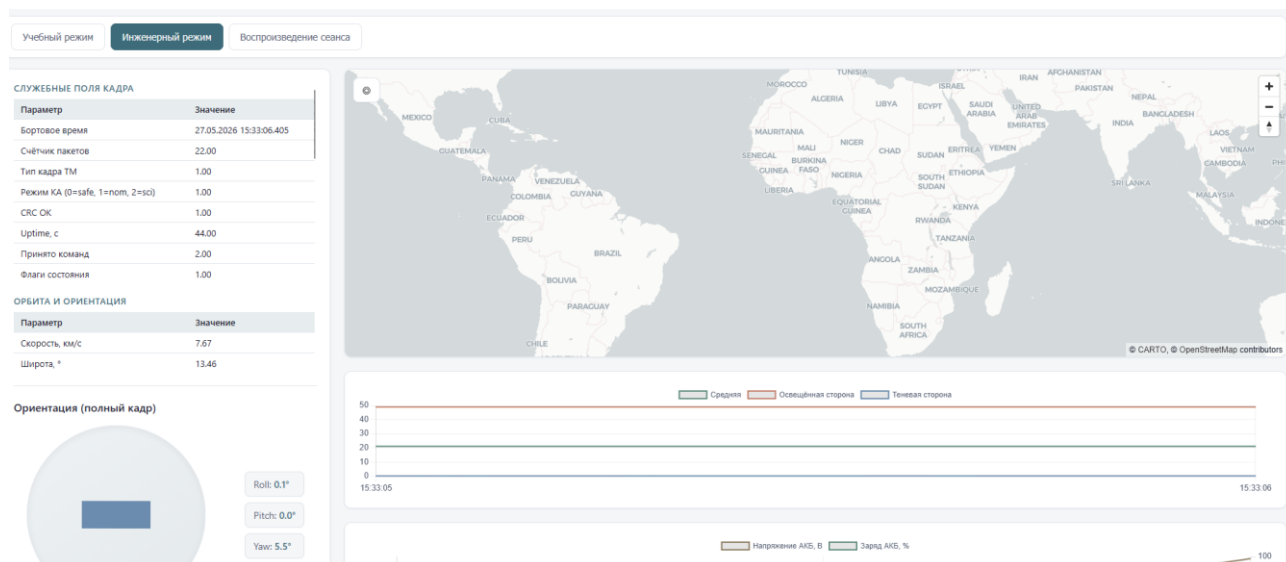


**Рис.13. Верхняя панель с кнопками выгрузок и переключателями**

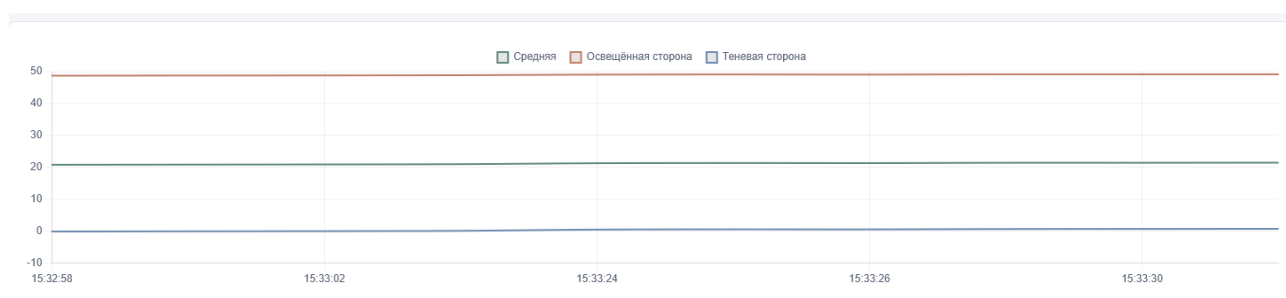


**Рис.14.1. Учебный режим**

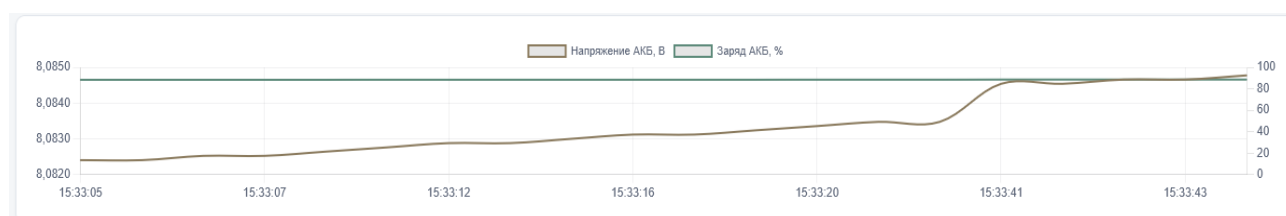
В верхней панели расположены кнопки «Скачать PDF», «CSV (учебный)», «CSV (полный)», «JSON сеансы», а также переключатель темы оформления и выпадающий список языка. Кнопки выгрузок открывают соответствующие файлы в новой вкладке браузера.



**Рис.14.2. Инженерный режим**



**Рис.14.3. График температур в учебном режиме**



**Рис.14.4. График электропитания в инженерном режиме**

Графики обновляются в реальном времени по мере поступления новых кадров телеметрии. Для очистки графиков достаточно перезагрузить страницу.

## Режим воспроизведения сеанса:

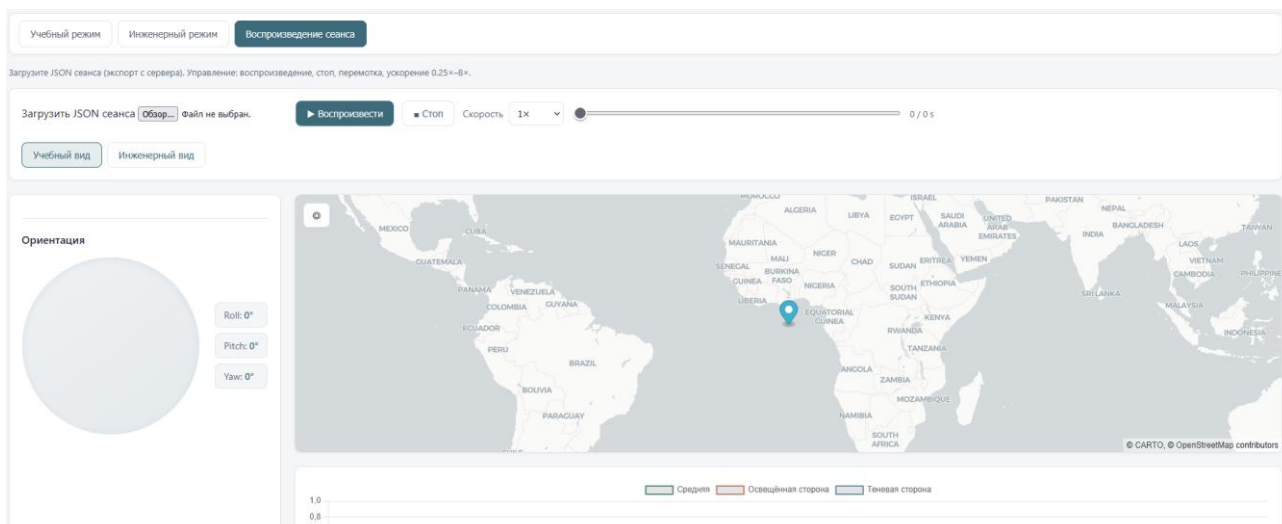


Рис.15.1. Вкладка «Воспроизведение сеанса»

Для перехода в режим воспроизведения нужно нажать на вкладку «Воспроизведение сеанса». В этом режиме можно загрузить ранее экспортированный JSON-файл сеанса с помощью кнопки выбора файла. После выбора файла приложение разбирает его и подготавливает к воспроизведению.

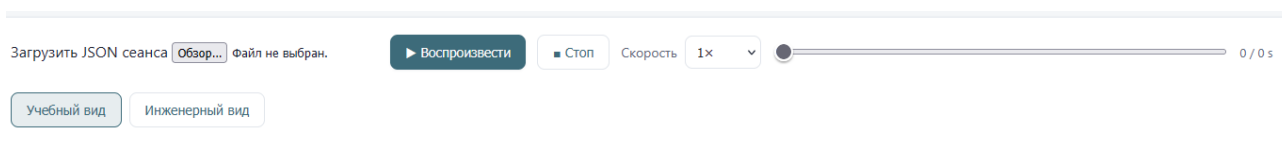


Рис.15.2. Управление воспроизведением

Если файл загружен корректно, в интерфейсе появятся данные первого кадра. Управление воспроизведением осуществляется кнопками «Воспроизвести» и «Стоп», ползунком перемотки и выпадающим списком скорости. Для каждого кадра отображается следующее:

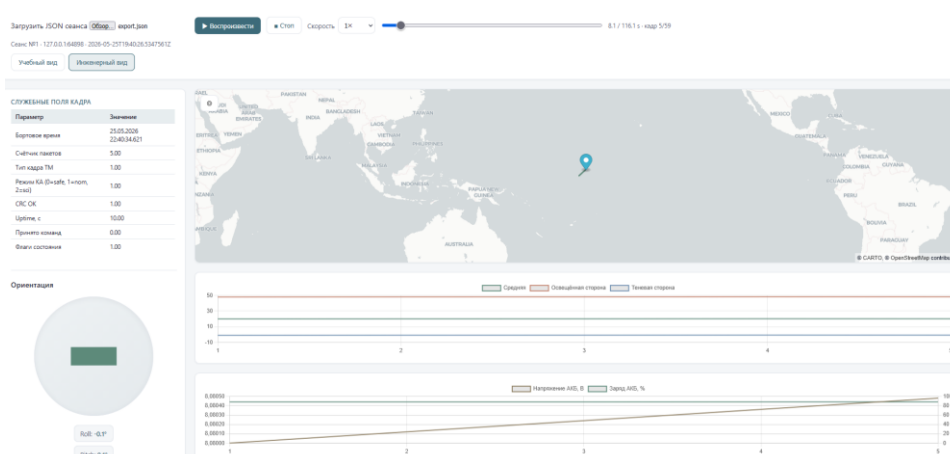
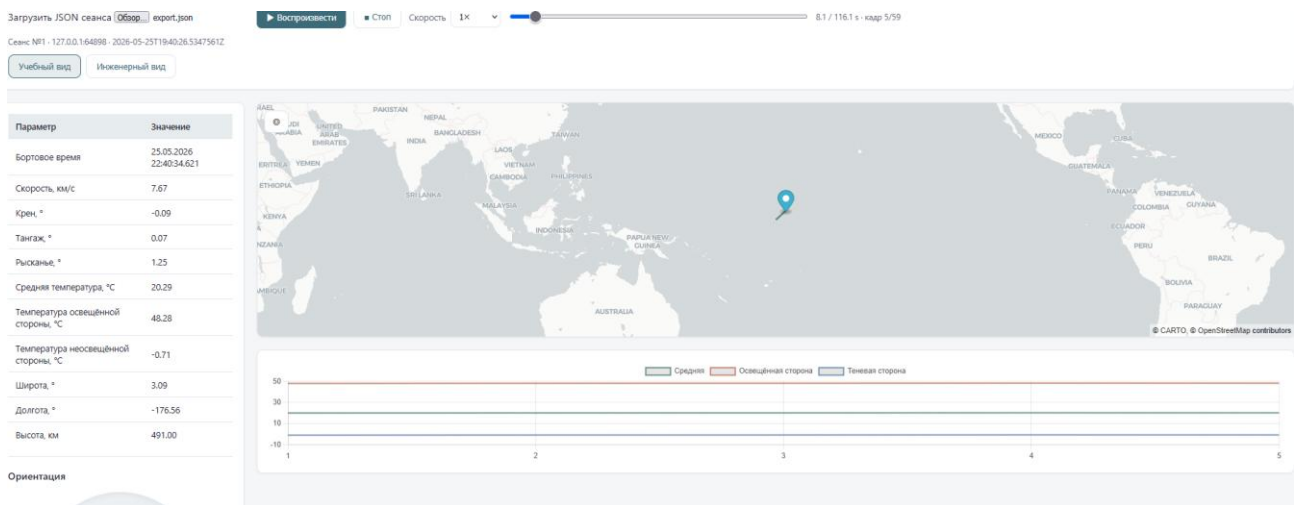


Рис.16.3. Подробное отображение кадра при воспроизведении

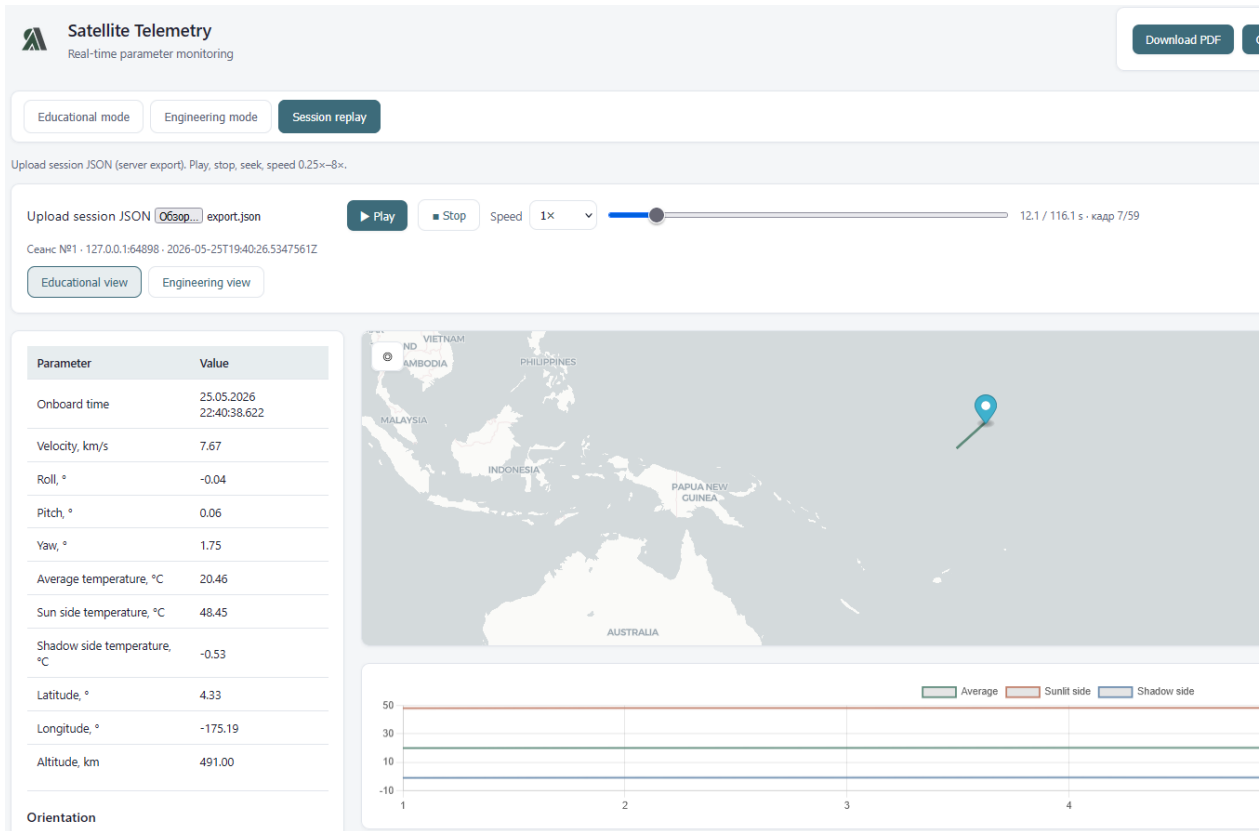
Для просмотра трека и графиков по всем кадрам сеанса используется ползунок перемотки. Чтобы переключить режим отображения, используйте кнопки «Учебный вид» и «Инженерный вид» в шапке режима воспроизведения.



**Рис.17.4. Переключение учебного и инженерного вида**

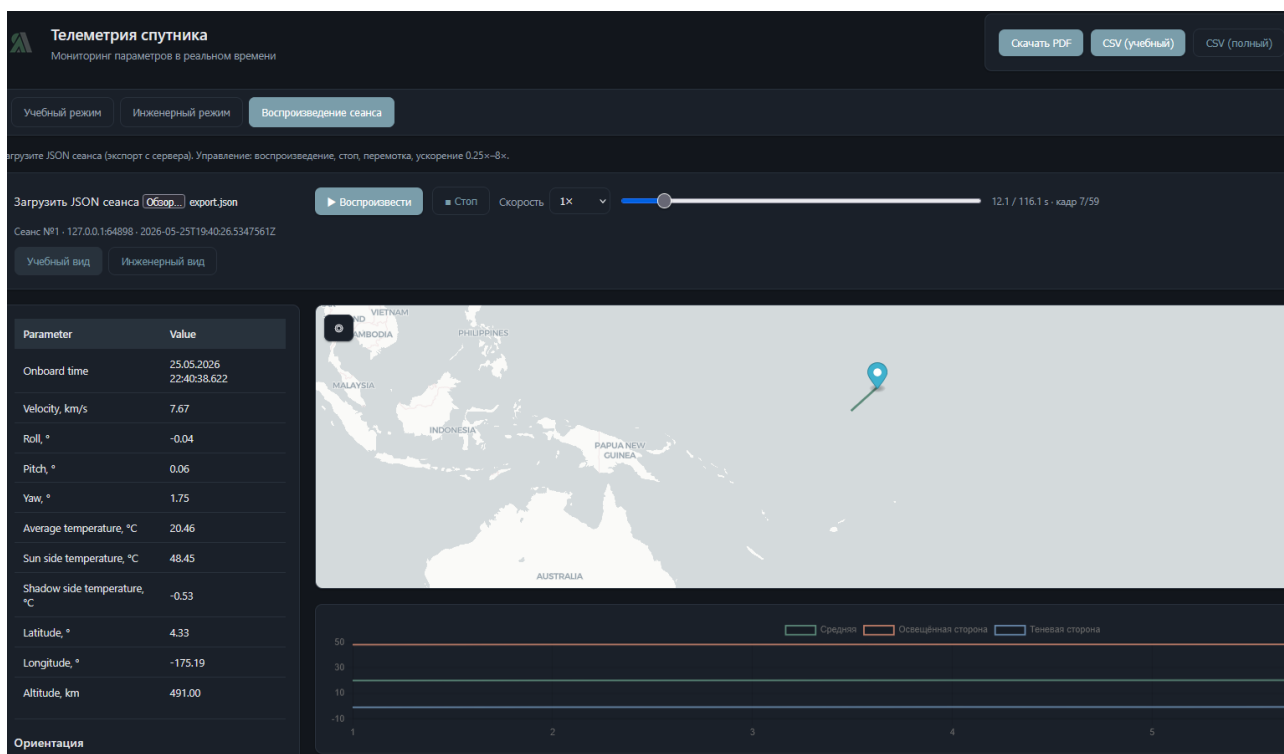
На странице воспроизведения карта показывает пройденный путь до текущего кадра, а графики — динамику параметров до текущего кадра. Изменение скорости применяется сразу после её выбора в выпадающем списке.

**Дополнительные настройки:**



**Рис.18.1. Переключение языка интерфейса**

В верхней панели доступен выпадающий список языка интерфейса. Поддерживаются русский и английский языки; изменения применяются ко всем меткам, подсказкам и заголовкам мгновенно, без перезагрузки страницы.



**Рис.18.2. Переключение темы оформления**

Кнопка «Тёмная тема» / «Светлая тема» в верхней панели переключает оформление приложения. Состояние темы сохраняется в локальном хранилище браузера и восстанавливается при следующем открытии страницы.

Сервер по умолчанию слушает HTTP-порт 8080 для веб-интерфейса и TCP-порт 9000 для приёма телеметрии. Эти параметры заданы в исходном коде серверной части; для их изменения требуется пересборка сервера из исходников.

Функциональные возможности веб-приложения полностью покрывают сценарии работы с телеметрией: просмотр в реальном времени, переключение режимов отображения, воспроизведение записанного сеанса, выгрузка отчётов (PDF/CSV/JSON), а также переключение языка и темы оформления. [28].

Веб-приложение реализовано с учётом особенностей работы в современных браузерах и поддерживает адаптивное отображение под различные разрешения экранов. Таким образом, веб-интерфейс обеспечивает полноценный

доступ к функционалу приёма и анализа телеметрии независимо от устройства, на котором осуществляется доступ. [18].

### 3.4. Руководство администратора

Для успешного развёртывания и работы системы на операционной системе Windows 10/11 (или Windows Server) необходимо выполнить следующие шаги:

1. Скачать и установить Go версии 1.20 или выше с официального сайта (<https://golang.org>). После установки убедиться, что переменная среды PATH содержит путь к каталогу bin Go.
2. Установить систему контроля версий Git с официального сайта (<https://git-scm.com>).
3. Клонировать репозиторий проекта в рабочую папку командой: `git clone <url репозитория>`.
4. Перейти в каталог `back` и получить зависимости проекта командой: `go mod download`.
5. Перейти в каталог `back/cmd/server` и проверить сборку командой: `go run main.go`.
6. Для сборки исполняемого файла выполнить из каталога `back`: `go build -o telemetry-server.exe ./cmd/server`.
7. Убедиться, что порты 8080 (HTTP) и 9000 (TCP) не заняты другими приложениями (`netstat -an | findstr 8080`).
8. Запустить полученный исполняемый файл `telemetry-server.exe`. В консоли появится сообщение `HTTP server listening on :8080`.
9. Открыть веб-браузер и перейти по адресу `http://localhost:8080` для проверки веб-интерфейса.
10. Для проверки приёма телеметрии подключиться к TCP-порту 9000 любым клиентом (например, генератором из репозитория).
11. Проверить выгрузки через адреса: `/api/telemetry/pdf`, `/api/telemetry/csv`, `/api/telemetry/csv/full`, `/api/telemetry/sessions/export`.

12. Убедиться, что папка `front` доступна серверу относительно рабочего каталога (для отдачи статических файлов веб-интерфейса).

13. Для остановки сервера используется сочетание `Ctrl+C` в окне терминала.

Для запуска вспомогательного генератора тестовой телеметрии (для проверки сервера без реального источника) необходимо выполнить следующие шаги:

#### **Настройка и запуск генератора:**

1. Перейти в каталог `generator` из корня репозитория.
2. Открыть терминал в каталоге `generator` и выполнить: `go mod download` (если требуется).
3. Запустить генератор командой: `go run main.go` (либо предварительно собрать командой `go build`).
4. Генератор подключается к TCP-порту 9000 локального сервера и начинает отправлять кадры с заданной частотой.
5. При необходимости настройки параметров генератора (адрес сервера, период отправки) изменить значения в исходном коде или передать через аргументы запуска.
6. Проверить, что в веб-интерфейсе появляются данные от подключения генератора в реальном времени.
7. Для остановки генератора используется сочетание `Ctrl+C` в окне терминала.
8. При необходимости можно одновременно запустить несколько экземпляров генератора — каждый создаст отдельный сеанс на сервере.

#### **Настройка веб-клиента:**

1. Дополнительная установка веб-клиента не требуется: статические файлы отдаются сервером.
2. Для редактирования веб-клиента достаточно текстового редактора (Visual Studio Code, Sublime Text и т. п.).

3. Файлы клиента расположены в каталоге front (HTML, CSS, JS, изображения, шрифты).
4. Скрипты подключаются последовательно в front/index.html — порядок подключения важен для зависимостей модулей.
5. Используемые внешние библиотеки (MapLibre GL JS, Chart.js) подключаются через CDN.
6. Для проверки локальной отладки рекомендуется открывать страницу через сервер; при открытии index.html напрямую WebSocket /ws работать не будет.
7. При изменении JS- или CSS-файлов достаточно обновить страницу в браузере (Ctrl+F5 для очистки кэша).
8. Для проверки работы клиента в чистом окружении рекомендуется использовать режим инкогнито браузера для исключения кэширования.
9. Для удалённого доступа к веб-интерфейсу необходимо открыть порт 8080 в межсетевом экране и убедиться в корректности сетевой маршрутизации.

### **Выводы:**

В процессе реализации проекта было проведено тестирование компонентов серверной и клиентской частей. Проверена корректность взаимодействия между приёмником TCP, ядром обработки, веб-клиентом и эндпоинтами выгрузок. Реализованы проверки разбора кадров, обработки ошибок и устойчивости к некорректным входным данным.

Выявленные ошибки были устранены, а пользовательский интерфейс адаптирован под основные сценарии просмотра телеметрии в реальном времени и анализа записанных сеансов.

Разработана техническая документация, включающая описание архитектуры, форматов экспорта и инструкций по запуску сервера и вспомогательного генератора. Результаты тестирования подтверждают стабильную и корректную работу всех ключевых модулей и готовность приложения к использованию на практике.



## ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было спроектировано и реализовано клиент-серверное приложение для приёма, обработки и визуализации телеметрических данных летательных аппаратов с использованием технологий веб-разработки и языка Go. Система ориентирована на оперативный приём кадров по TCP, передачу данных в веб-интерфейс в реальном времени и предоставление средств анализа записанных сеансов связи.

В процессе реализации выпускной работы были успешно решены поставленные задачи:

1. Проведён сравнительный анализ существующих систем мониторинга и оценена их применимость к задаче приёма телеметрии.
2. Определена оптимальная структура клиент-серверного приложения с разделением приёма, обработки, хранения и доставки данных.
3. Реализованы основные программные компоненты: TCP-приёмник, обработчик кадров, in-memory хранилище сеансов, WebSocket-доставка и HTTP-эндпоинты выгрузок (PDF, CSV, JSON).
4. Осуществлено тестирование ключевых модулей приложения, обеспечивших корректность разбора кадров, отображения данных и формирования отчётов;
5. Разработана сопроводительная документация, описывающая структуру, форматы данных и принцип работы системы.

В ходе выполнения работы были получены практические навыки построения клиент-серверных приложений для работы с потоковыми данными, освоены современные инструменты и технологии разработки: язык Go, библиотека Gorilla WebSocket, веб-технологии (HTML/CSS/JavaScript), Chart.js, MapLibre GL JS, форматы PDF/CSV/JSON для экспорта. Также были закреплены знания в области проектирования архитектуры серверных приложений, REST API и обеспечения удобного пользовательского интерфейса.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Donovan, A. A. The Go programming language / A. A. Donovan, B. W. Kernighan. — New York : Addison-Wesley, 2015. — 380 p. — ISBN 978-0-13-419044-0.
2. Кокс-Бьюкин, Дж. Идиоматический Go: введение в современный язык программирования / Дж. Кокс-Бьюкин. — Москва : ДМК Пресс, 2022. — 320 с. — ISBN 978-5-97060-928-9.
3. Калбертсон, Р. Программирование на Go. Разработка приложений XXI века / Р. Калбертсон, К. Браун, К. Кокс. — Москва : ДМК Пресс, 2020. — 432 с. — ISBN 978-5-97060-820-6.
4. The Go Programming Language Specification [Электронный ресурс] / The Go Authors. — Mountain View : Google, 2024. — URL: <https://go.dev/ref/spec> (дата обращения: 12.05.2026).
5. Effective Go [Электронный ресурс] / The Go Authors. — Mountain View : Google, 2024. — URL: [https://go.dev/doc/effective\\_go](https://go.dev/doc/effective_go) (дата обращения: 12.05.2026).
6. Документация пакета net/http [Электронный ресурс] / The Go Authors. — URL: <https://pkg.go.dev/net/http> (дата обращения: 14.05.2026).
7. Gorilla WebSocket. Documentation [Электронный ресурс] / Gorilla Web Toolkit. — URL: <https://pkg.go.dev/github.com/gorilla/websocket> (дата обращения: 14.05.2026).
8. Fette, I. The WebSocket Protocol : RFC 6455 [Электронный ресурс] / I. Fette, A. Melnikov. — IETF, 2011. — URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата обращения: 14.05.2026).
9. Документация пакета encoding/binary [Электронный ресурс] / The Go Authors. — URL: <https://pkg.go.dev/encoding/binary> (дата обращения: 14.05.2026).
10. gofpdf : Go PDF generator [Электронный ресурс] / J. Adamson. — URL: <https://pkg.go.dev/github.com/jung-kurt/gofpdf> (дата обращения: 14.05.2026).

11. Шеффер, К. Создание реактивных одностраничных приложений на JavaScript / К. Шеффер, Б. Купер. — Санкт-Петербург : Питер, 2022. — 624 с. — ISBN 978-5-4461-1903-2.
12. Хавербеке, М. Выразительный JavaScript. Современное веб-программирование / М. Хавербеке. — 3-е изд. — Санкт-Петербург : Питер, 2020. — 656 с. — ISBN 978-5-4461-1226-2.
13. Скляр, Д. Изучаем JavaScript : руководство по созданию современных веб-сайтов / Д. Скляр. — 3-е изд. — Москва : Эксмо, 2024. — 720 с. — ISBN 978-5-04-178452-5.
14. HTML Living Standard [Электронный ресурс] / WHATWG. — URL: <https://html.spec.whatwg.org> (дата обращения: 16.05.2026).
15. Cascading Style Sheets (CSS) Snapshot 2024 [Электронный ресурс] / W3C. — URL: <https://www.w3.org/TR/CSS> (дата обращения: 16.05.2026).
16. MDN Web Docs. The WebSocket API [Электронный ресурс] / Mozilla. — URL: [https://developer.mozilla.org/ru/docs/Web/API/WebSockets\\_API](https://developer.mozilla.org/ru/docs/Web/API/WebSockets_API) (дата обращения: 16.05.2026).
17. Chart.js Documentation [Электронный ресурс] / Chart.js Contributors. — URL: <https://www.chartjs.org/docs/latest> (дата обращения: 17.05.2026).
18. MapLibre GL JS Documentation [Электронный ресурс] / MapLibre Project. — URL: <https://maplibre.org/maplibre-gl-js/docs> (дата обращения: 17.05.2026).
19. Visual Studio Code Documentation [Электронный ресурс] / Microsoft. — URL: <https://code.visualstudio.com/docs> (дата обращения: 18.05.2026).
20. Чакон, С. Pro Git / С. Чакон, Б. Штрауб. — 2-е изд. — Москва : Манн, Иванов и Фербер, 2019. — 456 с. — ISBN 978-5-00146-241-2.
21. Танненбаум, Э. Компьютерные сети / Э. Танненбаум, Д. Уэзеролл. — 6-е изд. — Санкт-Петербург : Питер, 2024. — 960 с. — ISBN 978-5-4461-1949-0.

22. Куроуз, Дж. Компьютерные сети : нисходящий подход / Дж. Куроуз, К. Росс. — 8-е изд. — Москва : Эксмо, 2022. — 912 с. — ISBN 978-5-04-122336-4.
23. Хилл, Дж. Программирование на C++ для встроенных систем и систем телеметрии / Дж. Хилл. — Москва : ДМК Пресс, 2021. — 528 с. — ISBN 978-5-97060-882-4.
24. Ларман, К. Применение UML 2.0 и шаблонов проектирования : практическое руководство / К. Ларман. — 3-е изд. — Москва : Вильямс, 2019. — 736 с. — ISBN 978-5-907144-32-3.
25. Райнхейт, Р. RESTful Web APIs / Р. Райнхейт, Л. Ричардсон, М. Амундсен. — Москва : ДМК Пресс, 2018. — 408 с. — ISBN 978-5-97060-595-3.
26. Newman, S. Building Microservices / S. Newman. — 2nd ed. — Sebastopol : O'Reilly Media, 2021. — 612 p. — ISBN 978-1-4920-3402-5.
27. ГОСТ Р 56939–2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования. — Москва : Стандартинформ, 2024. — 24 с.
28. ECSS-E-ST-70-41C. Telemetry and telecommand packet utilization [Электронный ресурс] / European Cooperation for Space Standardization. — Noordwijk : ESA, 2016. — URL: <https://ecss.nl> (дата обращения: 20.05.2026).
29. CCSDS 133.0-B-2. Space packet protocol [Электронный ресурс] / The Consultative Committee for Space Data Systems. — Washington : CCSDS, 2020. — URL: <https://public.ccsds.org/Pubs/133x0b2.pdf> (дата обращения: 20.05.2026).
30. PostgreSQL 16 Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. — URL: <https://www.postgresql.org/docs/16> (дата обращения: 22.05.2026).

## ПРИЛОЖЕНИЕ А

Графическая часть выпускной квалификационной работы.

В графическую часть выпускной квалификационной работы входят:

1. Актуальность.
2. Цель и задачи.
3. Функциональные требования к системе.
4. Описание приложения
5. Анализ существующих решений (Prometheus, Zabbix).
6. Инструменты разработки приложения.
7. Архитектура приложения.
8. Архитектура клиентской части.
9. Архитектура серверной части.
10. Диаграмма действий пользователя.
11. Диаграмма классов.
12. Контекстная диаграмма IDEF0.
13. Диаграмма декомпозиции IDEF0.
15. Учебный режим веб-приложения.
16. Инженерный режим веб-приложения.
17. Режим воспроизведения сеанса.
18. Тёмный режим оформления.
19. Заключение.

## Актуальность

Актуальность работы обусловлена ростом потребности в простых и доступных инструментах для работы с телеметрическими данными, обеспечивающих их приём, обработку и наглядную визуализацию в реальном времени. Современные решения часто ориентированы на промышленное применение и обладают высокой сложностью настройки и эксплуатации, что ограничивает круг пользователей, способных эффективно с ними работать.

### 1. Актуальность.

## Цель и задачи

### Цель

Разработка клиент-серверного приложения для мониторинга и визуализации телеметрии летательных аппаратов с поддержкой экспорта и импорта отчётов.

### Задачи

- Провести сравнительный анализ существующих подходов и систем аналогичного назначения.
- Определить оптимальную структуру приложения и протокол обмена данными.
- Реализовать программные компоненты системы приёма, обработки, хранения и визуализации телеметрии.
- Реализовать простой и интуитивный интерфейс
- Обеспечить безопасное хранение данных
- Осуществить тестирование компонентов системы и проверку корректности обработки данных

### 2. Цели и задачи.

## Требования к системе

### Функциональные

- Получение бинарных кадров телеметрии.
- Разбор полученной телеметрии и её визуализация.
- Выгрузка полученной телеметрии в файл.

### Нефункциональные

- Кроссплатформенность (Windows и Linux).
- Устойчивость к сбоям.
- Интуитивный интерфейс.
- Безопасность хранения данных.

## 3. Функциональные требования к системе.

## Веб-приложение визуализации телеметрии

### Описание приложения

Приложение для просмотра телеметрии летательных аппаратов с целью изучения, документирования и наглядной демонстрации процесса полёта аппарата

### Принцип работы

Приложение связывается с станцией приёма телеметрии и осуществляет TCP подключение с последующей передачей бинарных кадров телеметрии, которая в последствии интерпретируется для записи и визуализации

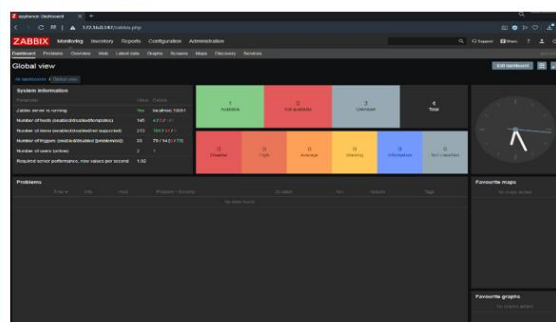
## 4. Описание приложения.

## Анализ аналогов и прототипов

### Prometheus



### Zabbix



## 5. Анализ существующих решений (Prometheus, Zabbix).

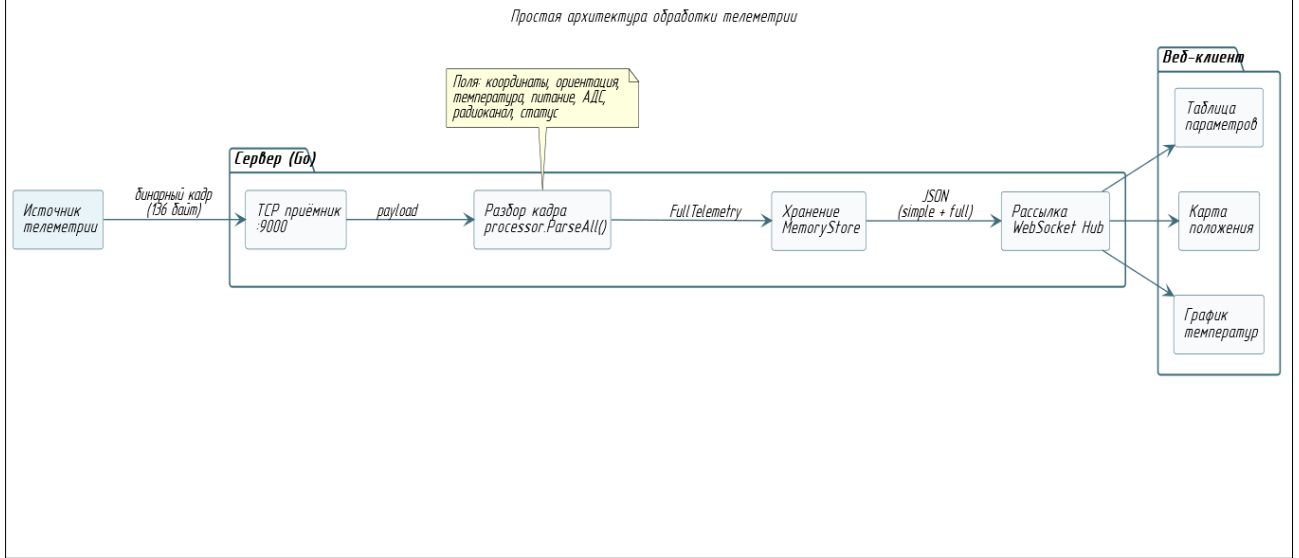
## Инструменты разработки приложения



## 6. Инструменты разработки приложения.

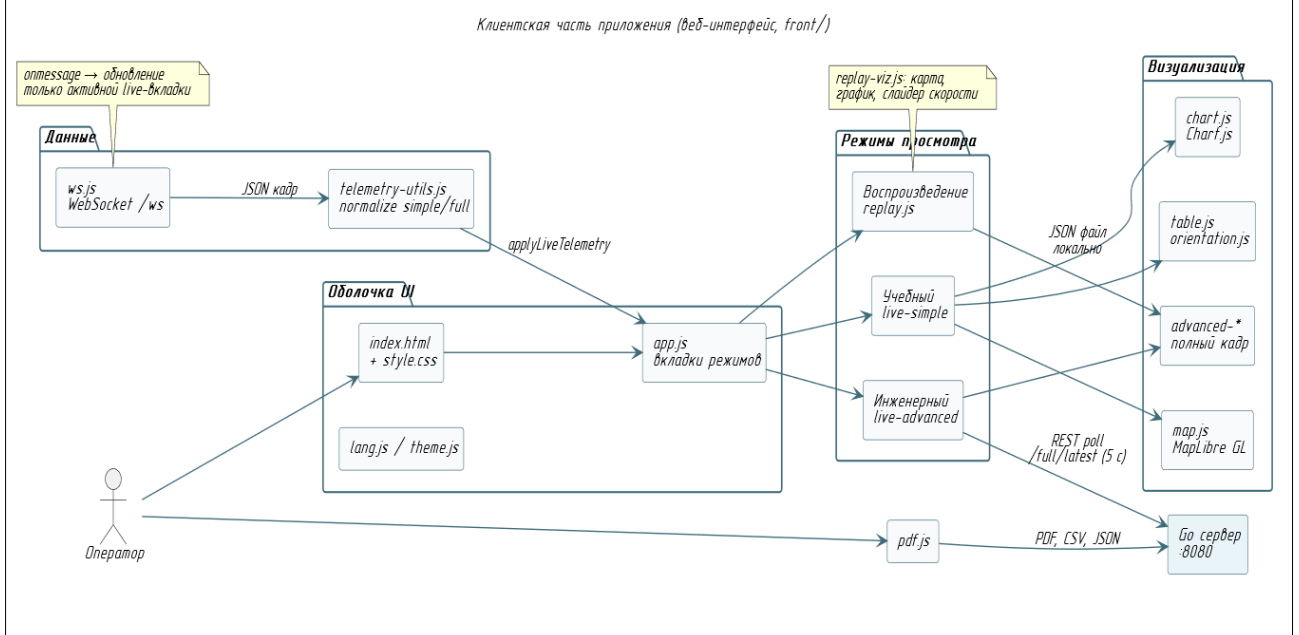


## Архитектура приложения



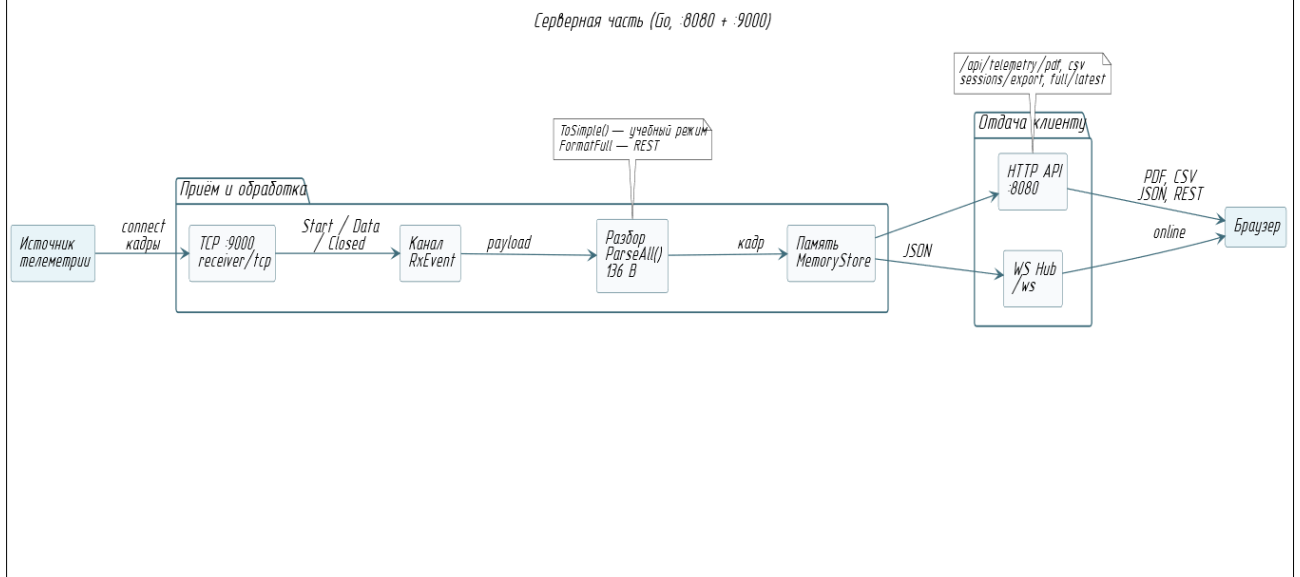
### 7. Архитектура приложения.

## Архитектура клиентской части



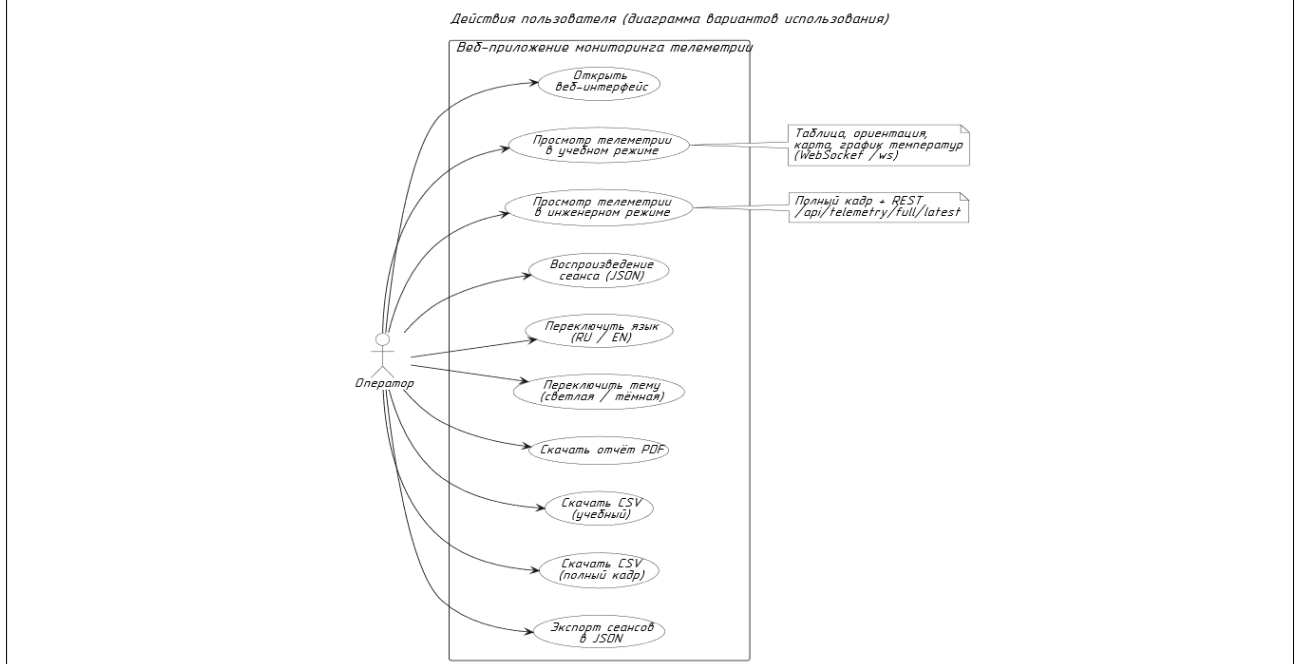
### 8. Архитектура клиентской части.

## Архитектура серверной части

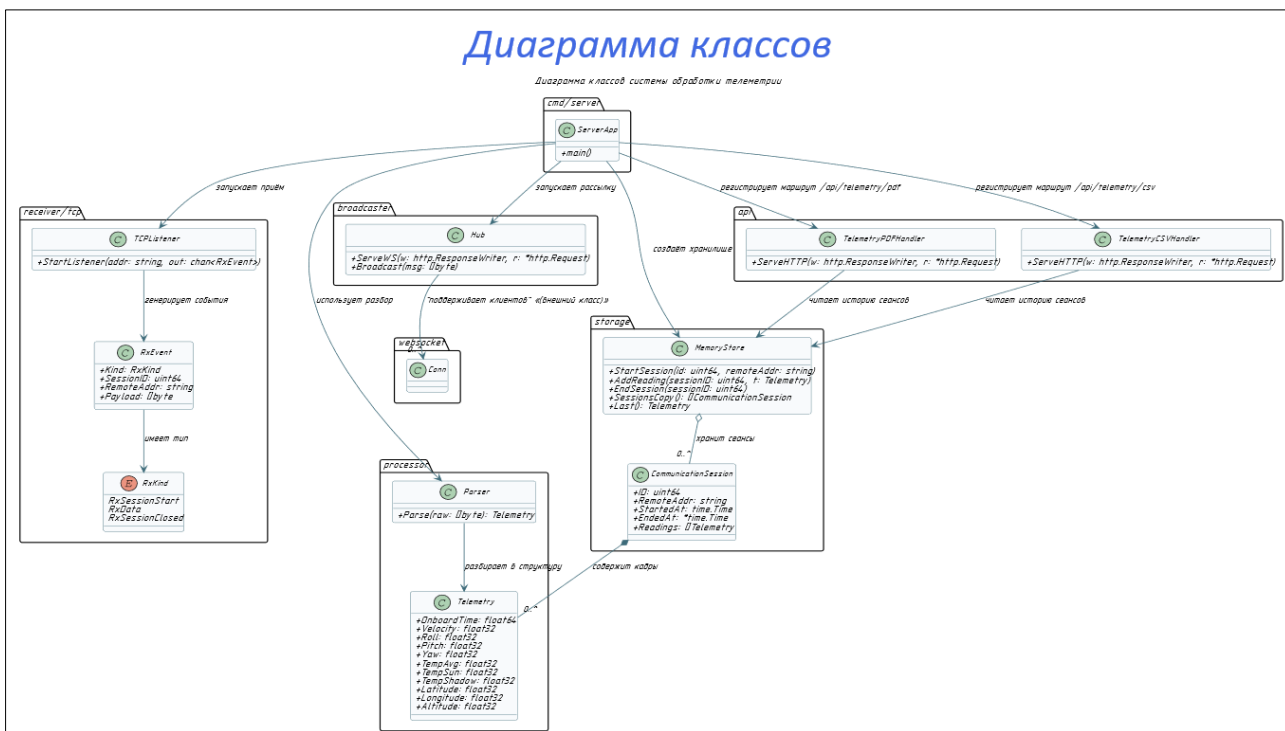


### 9. Архитектура серверной части.

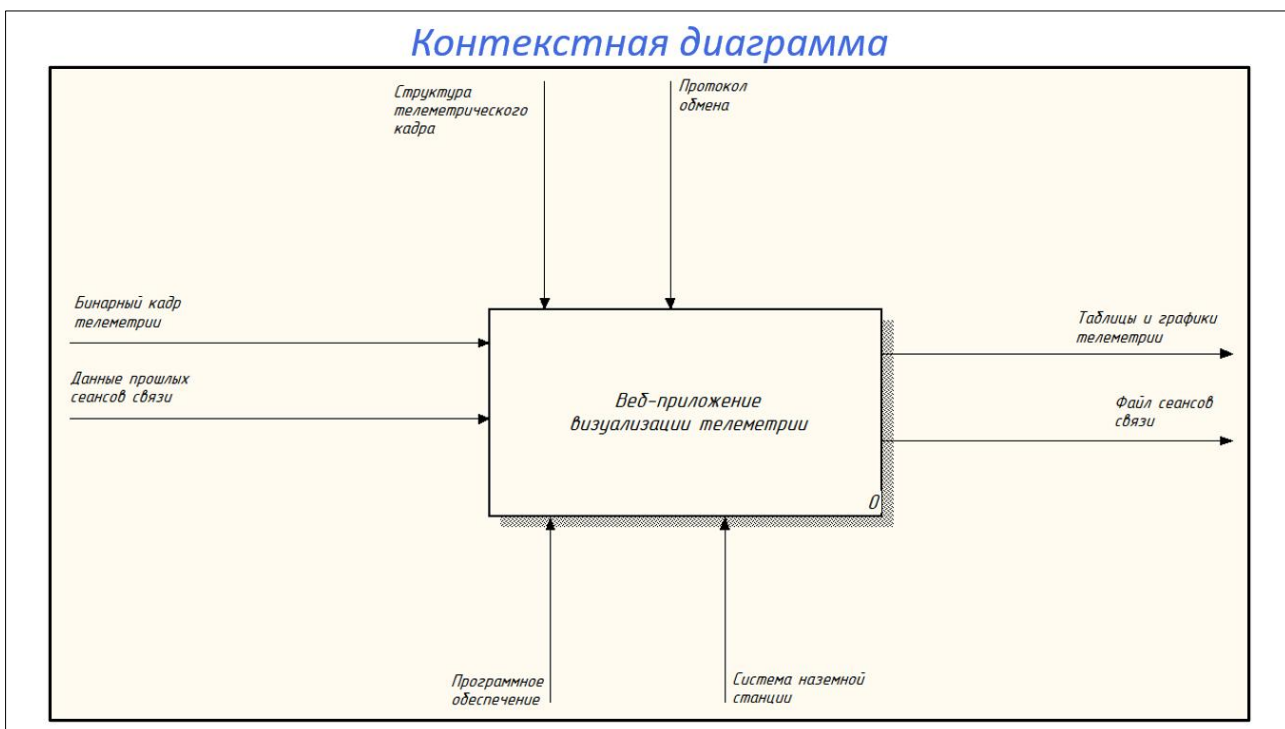
## Диаграмма действий пользователя



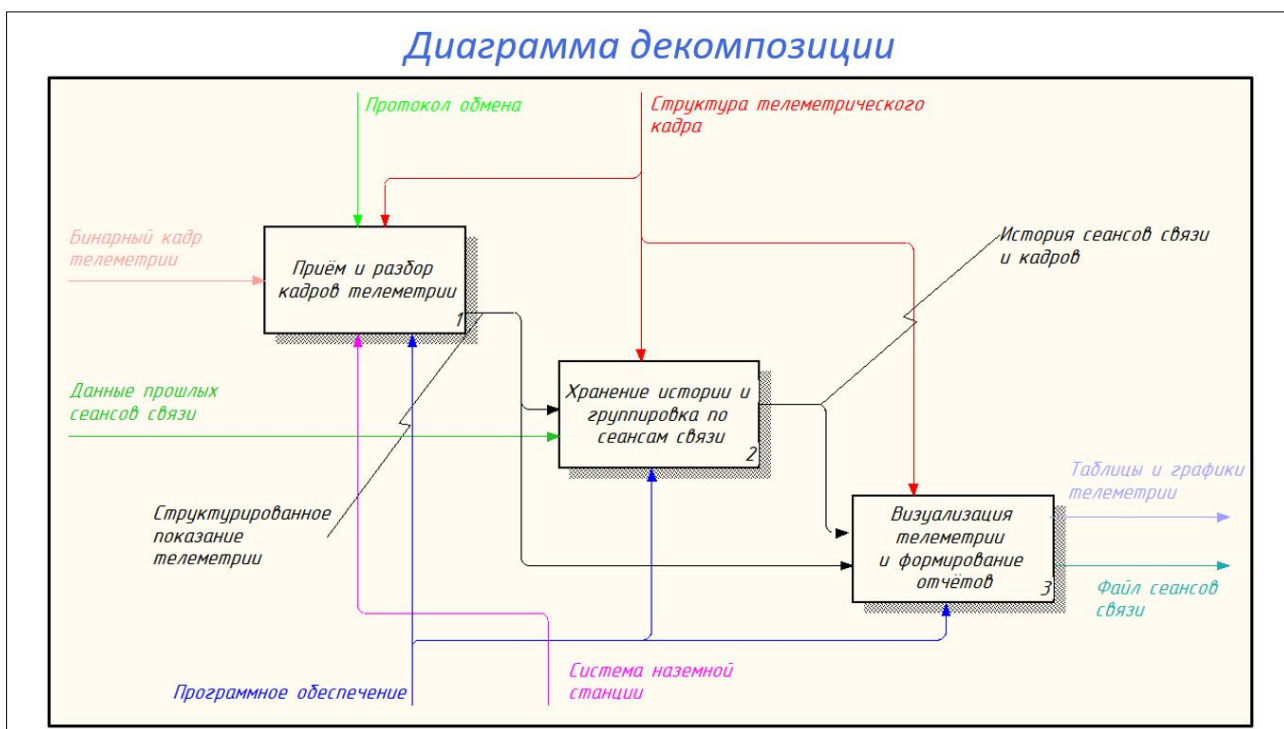
### 10. Диаграмма действий пользователя.



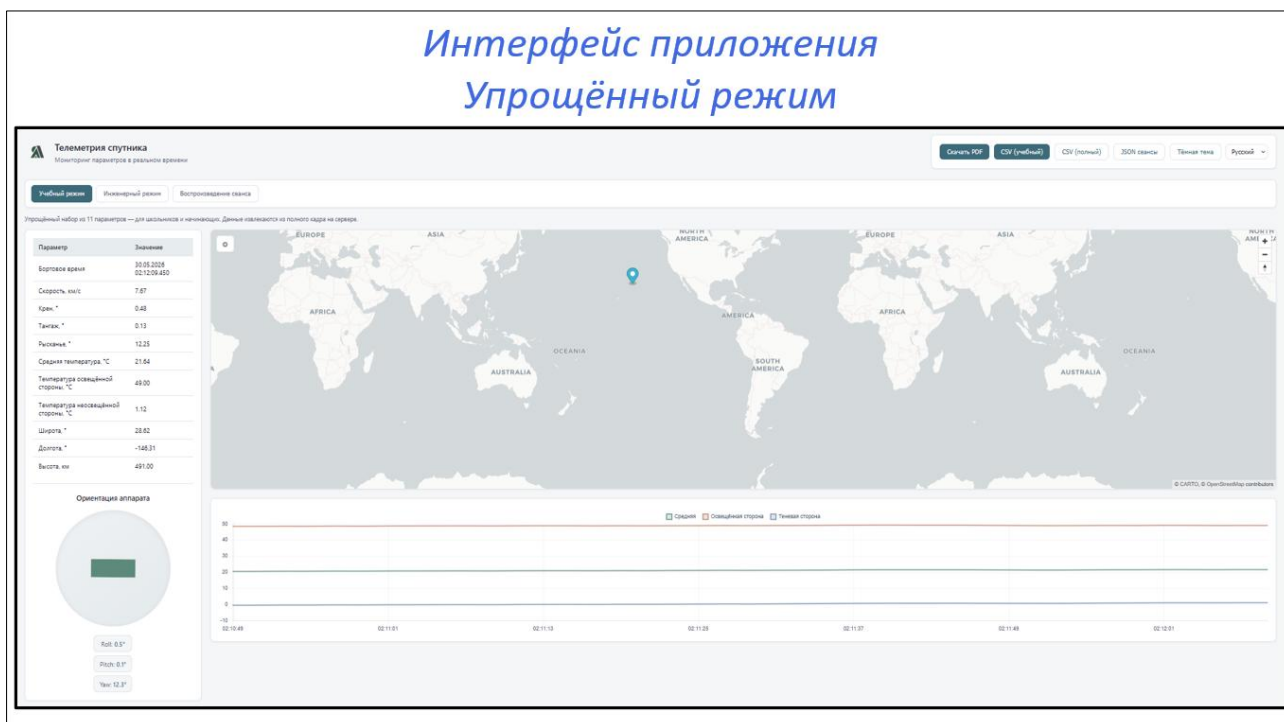
**11. Диаграмма классов.**



**12. Контекстная диаграмма IDEF0.**

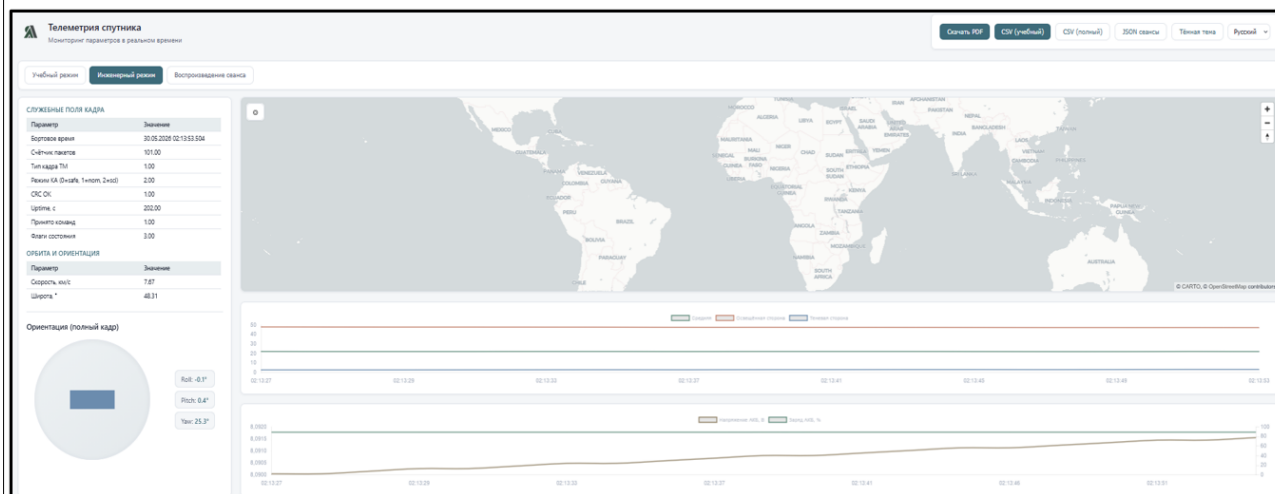


13. Диаграмма декомпозиции IDEF0.



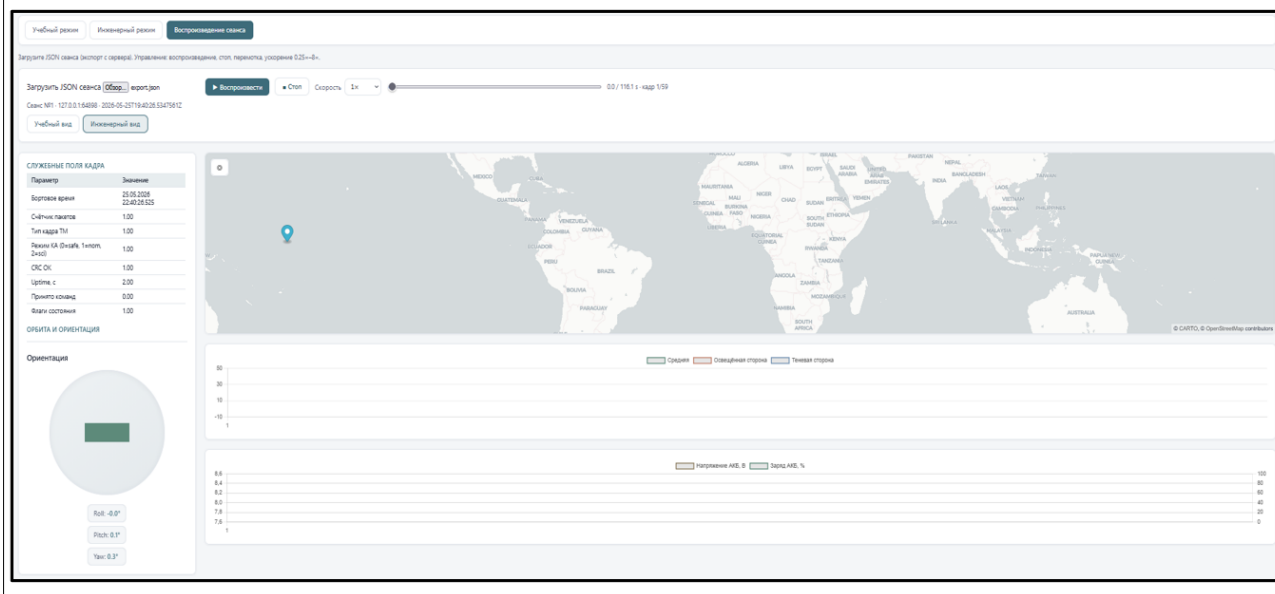
15. Учебный режим веб-приложения.

## Интерфейс приложения Инженерный режим



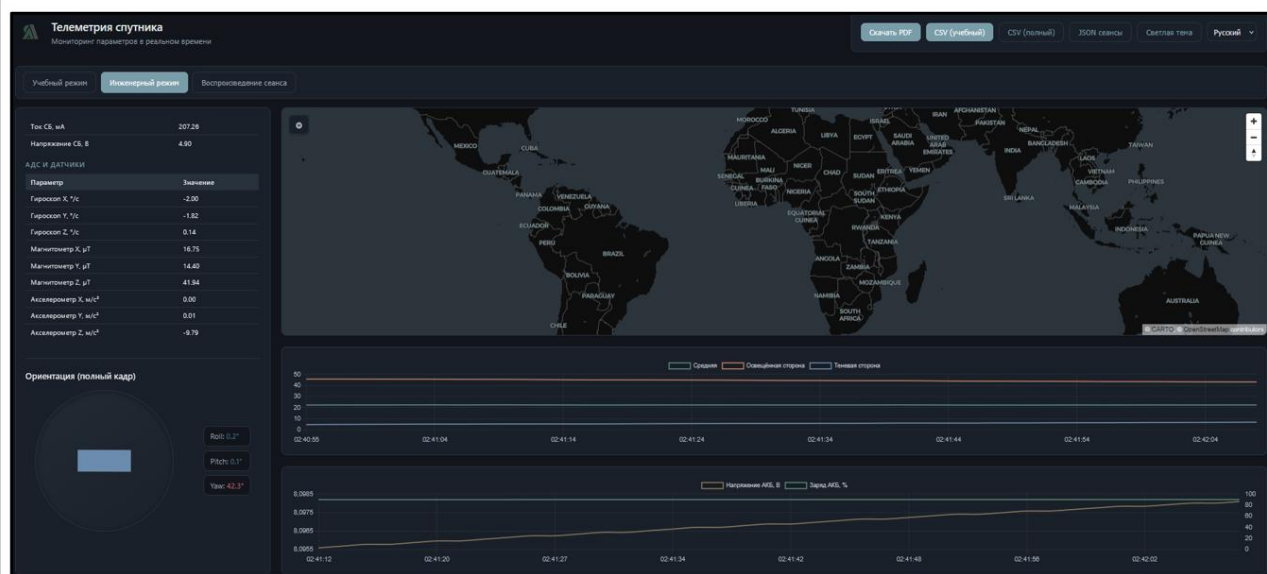
### 16. Инженерный режим веб-приложения.

## Интерфейс приложения Режим воспроизведения



### 17. Режим воспроизведения сеанса.

## Интерфейс приложения Тёмный режим оформления



### 18. Тёмный режим оформления.

## Заключение

Спроектировано и реализовано клиент-серверное приложение для приёма, обработки и визуализации телеметрических данных летательных аппаратов.

Выполнены следующие задачи:

- ☐ Определена оптимальная структура клиент-серверного приложения с разделением приёма, обработки, хранения и доставки данных;
- ☐ Реализованы основные программные компоненты и интуитивный интерфейс;
- ☐ Осуществлено тестирование ключевых модулей приложения, обеспечивших корректность разбора кадров, формирования отчётов, отображения и безопасности данных;

### 19. Заключение.